

# mmai\_HW2

March 5, 2026

## 0.1 SAVE A COPY OF THIS NOTEBOOK TO PUT ANSWERS INTO

#Part 1 (Reading Assignment), 20 points

**Reminder: there are 5 reading assignments, 3% each for 15% total of your final grade.**

The reading assignment consists of two papers - [one on alignment before fusion](#), and one on the [Platonic Representation Hypothesis](#). Read both papers and then answer the following questions:

1. Explain the implications of ‘align before fuse’ on your tasks of interest. at which degree, and what type of fusion needs to be performed? And what level of alignment would you need to perform for your data, so that subsequent fusion or representation learning is successful?
2. How can you perform controlled experiments to validate the types of fusion and/or alignment needed for your tasks? What are some challenges you foresee in fusing and aligning your data?
3. Explain the implications of ‘platonic representation hypothesis’ on your tasks of interest. Do you believe alignment between modalities would automatically emerge as models trained on your data are scaled up?
4. What are some reasons why alignment would not emerge i.e., counter-arguments to the platonic representation hypothesis? You are encouraged to search for follow-up works to the original paper that both support and counter the original arguments.
5. What experiments would you propose to validate the existence and emergence of alignment in your tasks?
6. Can you also think of some downsides of strongly (or perfectly) aligning your data modalities? How can you design experiments to validate that these risks are not present in your trained models?

Answers: 1. For the task of estimating a speaker’s credibility from video and text, align before fuse means first mapping visual and textual features into a shared semantic space before combining them. This helps the model learn meaningful relationships between what the speaker says and their visual behavior. In this setting, intermediate or late fusion is appropriate: separate encoders process video and text, align their representations, and then a multimodal module fuses them. The required alignment is mainly semantic-level alignment, so that both modalities capture related signals about credibility before fusion. 2. To validate different fusion and alignment strategies, we can perform controlled experiments by: comparing several model variants, such as text-only, video-only, early fusion, and late or intermediate fusion models. We can also test models with and without explicit alignment objectives (e.g., contrastive alignment) and compare their performance on the credibility prediction task. One challenge is that video and text signals may not always correspond temporally or semantically, which can make alignment difficult. In addition, visual cues related to credibility can be subtle and noisy, making effective multimodal fusion more challenging.

3. For the task of estimating a speaker’s credibility from video and text, the Platonic Representation Hypothesis suggests that as models and datasets scale, representations from different modalities may naturally converge toward a shared semantic structure. This means that visual cues (e.g., facial expressions or gestures) and textual content may increasingly align in the learned representation space. However, in this task alignment may not fully emerge automatically, since credibility signals can be subtle and weakly correlated across modalities. Therefore, while scaling models may improve cross-modal consistency, explicit alignment methods or multimodal training objectives may still be needed to ensure reliable fusion and representation learning.
4. Here are some reasons:
  - First, different modalities contain different information structures. Recent work [<https://arxiv.org/pdf/2602.19367>] studying multimodal alignment across time series, vision, and language found that independently trained encoders can exhibit nearly orthogonal representation geometries without explicit alignment, suggesting convergence is not guaranteed.
  - Secondly, training objectives strongly influence representations. Studies [<https://arxiv.org/abs/2411.05561>] show that the objective function used during training can determine how similar learned representations are across models and datasets. If models are trained for different goals or tasks, their representations may diverge rather than converge.
  - Finally, subtle or task-specific signals may require explicit alignment methods. Some research [<https://arxiv.org/pdf/2507.01201>] proposes explicit alignment techniques (e.g., joint autoencoder or optimal transport alignment) because independently trained unimodal models often occupy separate representation spaces and require additional training to align them effectively.
5. To test whether alignment emerges, we can train models at different scales (e.g., small vs. large models) and measure how similar the video and text representations become. For example, we can evaluate cross-modal retrieval (matching videos with their transcripts) and representation similarity between modalities as the model scales. We can also compare models with and without explicit alignment losses (e.g., contrastive learning). If alignment naturally emerges, larger models should show better cross-modal matching and more similar representations even without strong alignment supervision.
6. Strongly aligning modalities can remove useful modality-specific information and make the model rely too heavily on “one signal”. For example, the model might overfit to textual cues and ignore visual signals, reducing robustness when one modality is noisy. To test these risks, we can run ablation experiments (text-only, video-only, and multimodal models), introduce modality noise, and perform counterfactual tests such as swapping transcripts between videos. If the model remains stable and still benefits from both modalities, the risks are likely limited.

#Part 2 (Homework Assignment), 100 points

**Reminder: there are 5 homework assignments, 7% each for 35% total of your final grade.**

For this assignment, we will finally begin playing with some of the concepts discussed in the class regarding multimodal modeling.

The first part will deal with Einsum and Tensors.

## 1 Problem 1: Tensors (5 points)

(5 pts) Let's start with tensors. A tensor represents an N-th dimensional array of numbers. In machine learning, they are used to represent data as they can efficiently represent complex data to train with. We traditionally use PyTorch as the package of choice to work with tensors. Fill in the code below with the right tensor operations. Feel free to consult the documentation and the PyTorch tutorials for help.

```
[ ]: import torch
mat_A = torch.rand(3, 2)
mat_B = torch.rand(2, 3)

[ ]: # Common PyTorch operations

# Adding
mat_C = mat_A + mat_A

# Transpose
mat_A_transpose = mat_A.T

# Matrix multiplication
mat_mult = torch.matmul(mat_A, mat_B)

# Element-wise multiplication
mat_mult_elm = mat_A * mat_A

# Create a tensor of size (4, 4) of ones
ones = torch.ones(4, 4)

# Compute mean of A
mean_A = torch.mean(mat_A)
```

## 2 Problem 2: Einsum (5 points)

(10 pts) Now let's proceed with Einsum. This is a powerful, compact notation used for expressing complex tensor operations on multi-dimensional arrays using a simple string of index labels.

Here is a quick example of using einsum to multiply two matrices.

```
[ ]: A = torch.rand(3, 2)
B = torch.rand(2, 3)

C = torch.einsum('ij,jk->ik', A, B)
print(C)
```

```
tensor([[0.5393, 0.3655, 0.7011],
        [0.5780, 0.3550, 0.7565],
        [0.0983, 0.4995, 0.0680]])
```

The labels provide a shorthand as to what operation to do. Think of the left index as what is before, and the right as to what the dimensions of the final product should look like.

Now use this to do the other possible operations:

```
[ ]: a = torch.rand(3, 1)
      b = torch.rand(3, 1)

      A = torch.rand(3, 2)
      B = torch.rand(2, 3)

      # Dot Product of a and b
      d_prod = torch.einsum('ij,ij->', a, b)

      # Transpose using vector b
      transpose = torch.einsum('ij->ji', b)

      # Summation (element-wise and column-wise of A)
      sum_element = torch.einsum('ij->', A)
      sum_column = torch.einsum('ij->j', A)

      # Diagonal of A
      diag = torch.einsum('ii->i', A[:2, :2])

      # Outer Product of A and B
      outer = torch.einsum('i,j->ij', a.squeeze(1), b.squeeze(1))

[ ]: # Tests to verify that operations were done correctly
      def to_list(t):
          return t.detach().cpu().tolist()

      def check_dot_product(ans, a, b):
          expected = sum(i[0] * j[0] for i, j in zip(to_list(a), to_list(b)))
          assert torch.allclose(ans, torch.tensor(expected))

      def check_transpose(ans, b):
          b_list = to_list(b)
          expected = [[row[i] for row in b_list] for i in range(len(b_list[0]))]
          assert torch.allclose(ans, torch.tensor(expected))

      def check_sum_element(ans, A):
          expected = sum(val for row in to_list(A) for val in row)
          assert torch.allclose(ans, torch.tensor(expected))

      def check_sum_column(ans, A):
          A_list = to_list(A)
          expected = [sum(row[i] for row in A_list) for i in range(len(A_list[0]))]
          assert torch.allclose(ans, torch.tensor(expected))
```

```
def check_diagonal(ans, A):
    A_list = to_list(A)
    expected = [A_list[i][i] for i in range(min(len(A_list), len(A_list[0])))]
    assert torch.allclose(ans, torch.tensor(expected))

def check_outer_product(ans, a, b):
    a_l, b_l = to_list(a.squeeze(1)), to_list(b.squeeze(1))
    expected = [[i * j for j in b_l] for i in a_l]
    assert torch.allclose(ans, torch.tensor(expected))
```

```
[ ]: check_dot_product(d_prod, a, b)
      check_transpose(transpose, b)
      check_sum_element(sum_element, A)
      check_sum_column(sum_column, A)
      check_diagonal(diag, A)
      check_outer_product(outer, a, b)
```

### 3 Problem 3: Unimodal Models (10 points)

We now explore unimodal models and multimodal fusion. For the first part we will work on the image and audio digit dataset AV-MNIST to do digit classification. To benchmark effectiveness, we will use the [Multibench](#) benchmark. First, we will clone the repo, and get the necessary packages and dataset.

**Note: MAKE SURE YOU SWITCH TO A GPU TO RUN THE MODELS. RUN-TIME -> CHANGE RUNTIME TYPE -> T4 GPU (or any other). Be mindful of Google's GPU limits based on what kind of account you own.**

Also, if you are a student you should be able to have Colab Pro for free if you don't already. Take advantage of that!

**THIS IS AN EXAMPLE, DO NOT BE RESTRICTED BY WHAT WE DO HERE WHEN YOU HAVE TO IMPLEMENT THIS FOR YOUR OWN DATASET.**

### 4 Getting repo

```
[ ]: !git clone https://github.com/pliang279/MultiBench.git
      %cd MultiBench
```

```
fatal: destination path 'MultiBench' already exists and is not an empty
directory.
/content/MultiBench
```

## 5 Getting AV-MNIST dataset

```
[ ]: !mkdir data
      !pip install gdown
      !pip install torch==2.3.1 torchvision==0.18.1 torchtex==0.18.0 torchaudio==2.3.
      ↪1
      !pip install memory_profiler
```

```
mkdir: cannot create directory 'data': File exists
Requirement already satisfied: gdown in /usr/local/lib/python3.12/dist-packages
(5.2.1)
Requirement already satisfied: beautifulsoup4 in /usr/local/lib/python3.12/dist-
packages (from gdown) (4.13.5)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-
packages (from gdown) (3.24.3)
Requirement already satisfied: requests[socks] in
/usr/local/lib/python3.12/dist-packages (from gdown) (2.32.4)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages
(from gdown) (4.67.3)
Requirement already satisfied: soupsieve>1.2 in /usr/local/lib/python3.12/dist-
packages (from beautifulsoup4->gdown) (2.8.3)
Requirement already satisfied: typing-extensions>=4.0.0 in
/usr/local/lib/python3.12/dist-packages (from beautifulsoup4->gdown) (4.15.0)
Requirement already satisfied: charset_normalizer<4,>=2 in
/usr/local/lib/python3.12/dist-packages (from requests[socks]->gdown) (3.4.4)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-
packages (from requests[socks]->gdown) (3.11)
Requirement already satisfied: urllib3<3,>=1.21.1 in
/usr/local/lib/python3.12/dist-packages (from requests[socks]->gdown) (2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in
/usr/local/lib/python3.12/dist-packages (from requests[socks]->gdown)
(2026.2.25)
Requirement already satisfied: PySocks!=1.5.7,>=1.5.6 in
/usr/local/lib/python3.12/dist-packages (from requests[socks]->gdown) (1.7.1)
Requirement already satisfied: torch==2.3.1 in /usr/local/lib/python3.12/dist-
packages (2.3.1)
Requirement already satisfied: torchvision==0.18.1 in
/usr/local/lib/python3.12/dist-packages (0.18.1)
Requirement already satisfied: torchtex==0.18.0 in
/usr/local/lib/python3.12/dist-packages (0.18.0)
Requirement already satisfied: torchaudio==2.3.1 in
/usr/local/lib/python3.12/dist-packages (2.3.1)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-
packages (from torch==2.3.1) (3.24.3)
Requirement already satisfied: typing-extensions>=4.8.0 in
/usr/local/lib/python3.12/dist-packages (from torch==2.3.1) (4.15.0)
Requirement already satisfied: sympy in /usr/local/lib/python3.12/dist-packages
```

(from torch==2.3.1) (1.14.0)

Requirement already satisfied: networkx in /usr/local/lib/python3.12/dist-packages (from torch==2.3.1) (3.6.1)

Requirement already satisfied: Jinja2 in /usr/local/lib/python3.12/dist-packages (from torch==2.3.1) (3.1.6)

Requirement already satisfied: fsspec in /usr/local/lib/python3.12/dist-packages (from torch==2.3.1) (2025.3.0)

Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.1.105 in /usr/local/lib/python3.12/dist-packages (from torch==2.3.1) (12.1.105)

Requirement already satisfied: nvidia-cuda-runtime-cu12==12.1.105 in /usr/local/lib/python3.12/dist-packages (from torch==2.3.1) (12.1.105)

Requirement already satisfied: nvidia-cuda-cupti-cu12==12.1.105 in /usr/local/lib/python3.12/dist-packages (from torch==2.3.1) (12.1.105)

Requirement already satisfied: nvidia-cudnn-cu12==8.9.2.26 in /usr/local/lib/python3.12/dist-packages (from torch==2.3.1) (8.9.2.26)

Requirement already satisfied: nvidia-cublas-cu12==12.1.3.1 in /usr/local/lib/python3.12/dist-packages (from torch==2.3.1) (12.1.3.1)

Requirement already satisfied: nvidia-cufft-cu12==11.0.2.54 in /usr/local/lib/python3.12/dist-packages (from torch==2.3.1) (11.0.2.54)

Requirement already satisfied: nvidia-curand-cu12==10.3.2.106 in /usr/local/lib/python3.12/dist-packages (from torch==2.3.1) (10.3.2.106)

Requirement already satisfied: nvidia-cusolver-cu12==11.4.5.107 in /usr/local/lib/python3.12/dist-packages (from torch==2.3.1) (11.4.5.107)

Requirement already satisfied: nvidia-cuspars-cu12==12.1.0.106 in /usr/local/lib/python3.12/dist-packages (from torch==2.3.1) (12.1.0.106)

Requirement already satisfied: nvidia-nccl-cu12==2.20.5 in /usr/local/lib/python3.12/dist-packages (from torch==2.3.1) (2.20.5)

Requirement already satisfied: nvidia-nvtx-cu12==12.1.105 in /usr/local/lib/python3.12/dist-packages (from torch==2.3.1) (12.1.105)

Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from torchvision==0.18.1) (2.0.2)

Requirement already satisfied: pillow!=8.3.\*,>=5.3.0 in /usr/local/lib/python3.12/dist-packages (from torchvision==0.18.1) (11.3.0)

Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from torchtext==0.18.0) (4.67.3)

Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (from torchtext==0.18.0) (2.32.4)

Requirement already satisfied: nvidia-nvjitlink-cu12 in /usr/local/lib/python3.12/dist-packages (from nvidia-cusolver-cu12==11.4.5.107->torch==2.3.1) (12.8.93)

Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from Jinja2->torch==2.3.1) (3.0.3)

Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests->torchtext==0.18.0) (3.4.4)

Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests->torchtext==0.18.0) (3.11)

Requirement already satisfied: urllib3<3,>=1.21.1 in

```

/usr/local/lib/python3.12/dist-packages (from requests->torchtext==0.18.0)
(2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in
/usr/local/lib/python3.12/dist-packages (from requests->torchtext==0.18.0)
(2026.2.25)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in
/usr/local/lib/python3.12/dist-packages (from sympy->torch==2.3.1) (1.3.0)
Requirement already satisfied: memory_profiler in
/usr/local/lib/python3.12/dist-packages (0.61.0)
Requirement already satisfied: psutil in /usr/local/lib/python3.12/dist-packages
(from memory_profiler) (5.9.5)

```

```
[ ]: from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

```
[ ]: !mkdir -p data
!tar -xzf /content/drive/MyDrive/avmnist.tar.gz -C data
```

```
[ ]: !cd data
```

|                 |                     |                      |                     |
|-----------------|---------------------|----------------------|---------------------|
| coverage.xml    | examples            | pretrained           | sphinx              |
| data            | fusions             | private_test_scripts | tests               |
| datasets        | images              | README.md            | training_structures |
| deprecated      | __init__.py         | requirements.txt     | tutorial.md         |
| environment.yml | LICENSE             | robustness           | unimodals           |
| eval_scripts    | objective_functions | special              | utils               |

```
[ ]: # 1. Path to the folder you untarred
data_dir = '/content/MultiBench/data/avmnist'

from datasets.avmnist.get_data import get_dataloader
traindata, validdata, testdata = get_dataloader(data_dir, batch_size=256)
```

```

/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py:558:
UserWarning: This DataLoader will create 8 worker processes in total. Our
suggested max number of worker in current system is 2, which is smaller than
what this DataLoader is going to create. Please be aware that excessive worker
creation might get DataLoader running slow or even freeze, lower the worker
number to avoid potential slowness/freeze if necessary.
  warnings.warn(_create_warning_msg(

```



## 6 Getting packages

```
[ ]: import torch
import torch.nn as nn
import sys
import os
import torch.optim as optim
from tqdm import tqdm
from unimodals.common_models import GRU, MLP, Sequential, Identity
from training_structures.Supervised_Learning import train, test
```

We will now start by creating, training, and testing unimodal models for each of the AV-MNIST modalities.

## 7 Audio

```
[ ]: class AudioModel(nn.Module):
    def __init__(self, input_dim=12544, hidden_dim=64, dropout_probability=0.2):
        super(AudioModel, self).__init__()
        self.conv = nn.Sequential(
            # Start with a stride of 2 to instantly cut data in half
            nn.Conv2d(1, 16, kernel_size=3, stride=2, padding=1), # 112 -> 56
            nn.ReLU(),
            nn.MaxPool2d(2), # 56 -> 28
            nn.Conv2d(16, 32, kernel_size=3, stride=2, padding=1), # 28 -> 14
            nn.ReLU(),
            nn.Flatten() # Only 6272 features now!
        )
        self.fc = nn.Linear(6272, 10)

    def forward(self, x):
        x = x.view(-1, 1, 112, 112)
        return self.fc(self.conv(x))
```

## 8 Image

```
[ ]: import torch.nn.functional as F

class ImageModel(nn.Module):
    def __init__(self, dropout_prob=0.3):
        super(ImageModel, self).__init__()

        # input: [batch, 1, 28, 28]
        self.conv1 = nn.Conv2d(in_channels=1, out_channels=32, kernel_size=3,
                                padding=1)
```

```

        self.conv2 = nn.Conv2d(in_channels=32, out_channels=64, kernel_size=3,
        ↪padding=1)

        self.pool = nn.MaxPool2d(kernel_size=2, stride=2) # Reduces size by half

        # After two poolings: 28 -> 14 -> 7
        # Final flattened size: 64 channels * 7 * 7
        self.fc = nn.Sequential(
            nn.Linear(64 * 7 * 7, 128),
            nn.ReLU(),
            nn.Dropout(dropout_prob),
            nn.Linear(128, 10)
        )

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))

        # Flatten all dimensions except batch
        x = torch.flatten(x, 1)
        return self.fc(x)

```

## 9 Training and Testing

We use cross-entropy due to this being a classification task

```

[ ]: import torch.nn as nn
import torch.optim as optim
from torch.amp import autocast, GradScaler

# We use a scalar here to reduce system RAM use (to avoid crashing the session)
↪while not impacting performance.
scaler = GradScaler()

def train_and_test_unimodal(model, train_loader, valid_loader, test_loader,
↪modality_idx, epochs=5, lr=1e-3):
    device = torch.device("cuda")
    model.to(device)

    # Use CrossEntropyLoss for a classification task
    criterion = nn.CrossEntropyLoss()
    optimizer = optim.Adam(model.parameters(), lr=lr, weight_decay=1e-3)

    best_valid_loss = float('inf')

    for epoch in range(epochs):

```

```

model.train()
train_loss = 0
for batch in train_loader:
    # batch[0] = images, batch[1] = audio
    x = batch[modality_idx].to(device).float()

    # Classification labels must be Long tensors, not Float
    y = batch[2].to(device).long().squeeze()

    optimizer.zero_grad()

    with autocast(device_type='cuda'):
        outputs = model(x)
        loss = criterion(outputs, y)

    scaler.scale(loss).backward()
    scaler.step(optimizer)
    scaler.update()
    train_loss += loss.item()

# --- Validation Phase ---
model.eval()
valid_loss = 0
correct = 0
total = 0
with torch.no_grad():
    for batch in valid_loader:
        x = batch[modality_idx].to(device).float()
        y = batch[2].to(device).long().squeeze()

        outputs = model(x)
        valid_loss += criterion(outputs, y).item()

        _, predicted = torch.max(outputs.data, 1)
        total += y.size(0)
        correct += (predicted == y).sum().item()

avg_train = train_loss / len(train_loader)
avg_valid = valid_loss / len(valid_loader)
accuracy = 100 * correct / total

if avg_valid < best_valid_loss:
    best_valid_loss = avg_valid
    torch.save(model.state_dict(), 'best_avmnist_model.pt')

print(f"Epoch {epoch}: Train Loss: {avg_train:.4f} | Valid Acc:␣
↪{accuracy:.2f}%")

```

```

# Final Testing follows the same logic (CrossEntropy + Index 2)
print("\n--- Final Evaluation Complete ---")
model.load_state_dict(torch.load('best_avmnist_model.pt'))
model.eval()
test_loss = 0
correct = 0
total = 0
with torch.no_grad():
    for batch in test_loader:
        x = batch[modality_idx].to(device).float()
        y = batch[2].to(device).long().squeeze()

        outputs = model(x)
        test_loss += criterion(outputs, y).item()

        _, predicted = torch.max(outputs.data, 1)
        total += y.size(0)
        correct += (predicted == y).sum().item()

test_accuracy = 100 * correct / total
test_loss /= len(test_loader)
print(f"Final Test Loss: {test_loss:.4f} | Test Accuracy: ␣
↪{test_accuracy:.2f}%")

```

## 10 Training and testing for each modality:

### 11 Audio:

```

[ ]: audio_model = AudioModel()
train_and_test_unimodal(audio_model, traindata, validdata, testdata, ␣
↪modality_idx=1, epochs=8, lr=2e-3)

```

```

Epoch 0: Train Loss: 2.1473 | Valid Acc: 37.72%
Epoch 1: Train Loss: 2.0259 | Valid Acc: 39.42%
Epoch 2: Train Loss: 2.0049 | Valid Acc: 40.24%
Epoch 3: Train Loss: 1.9960 | Valid Acc: 39.88%
Epoch 4: Train Loss: 1.9891 | Valid Acc: 40.38%
Epoch 5: Train Loss: 1.9819 | Valid Acc: 40.36%
Epoch 6: Train Loss: 1.9772 | Valid Acc: 40.98%
Epoch 7: Train Loss: 1.9739 | Valid Acc: 40.94%

```

```

--- Final Evaluation Complete ---
Final Test Loss: 2.0146 | Test Accuracy: 41.03%

```

## 12 Image:

```
[ ]: image_model = ImageModel()  
train_and_test_unimodal(image_model, traindata, validdata, testdata,   
    ↪modality_idx=0, epochs=5, lr=1e-3)
```

```
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py:558:  
UserWarning: This DataLoader will create 8 worker processes in total. Our  
suggested max number of worker in current system is 2, which is smaller than  
what this DataLoader is going to create. Please be aware that excessive worker  
creation might get DataLoader running slow or even freeze, lower the worker  
number to avoid potential slowness/freeze if necessary.
```

```
warnings.warn(_create_warning_msg(
```

```
Epoch 0: Train Loss: 1.1470 | Valid Acc: 67.70%  
Epoch 1: Train Loss: 0.9613 | Valid Acc: 68.22%  
Epoch 2: Train Loss: 0.9381 | Valid Acc: 67.94%  
Epoch 3: Train Loss: 0.9291 | Valid Acc: 68.26%  
Epoch 4: Train Loss: 0.9242 | Valid Acc: 68.46%
```

```
--- Final Evaluation Complete ---
```

```
Final Test Loss: 0.8982 | Test Accuracy: 64.14%
```

Answer the following questions:

1. (5 points) Try to get the best performance out of each model by playing around with hyperparameters (hint: you may have to playing around and even add additional arguments to the layers like dropout, look at the documentation and look into how we can improve performance). List the best performance you were able to get and the hyperparameters you used.
  2. (5 points) Compare the performances of each modality. What do these suggest to you? What could be done to get the worst performing ones to get closer to the best performing modality/model?
1. After tuning the models, the best performance was obtained by adjusting the learning rate, number of epochs, and adding dropout to reduce overfitting.
    - Audio model: Best Test Accuracy: ~41.12%, Hyperparameters: learning rate = 2e-3, epochs = 8, dropout = 0.2, optimizer = Adam, batch size = 256
    - Image model: Best Test Accuracy: ~68.22%, Hyperparameters: learning rate = 1e-3, epochs = 5, dropout = 0.3, optimizer = Adam, batch size = 256
  2. The image model performs better than the audio model, suggesting that visual digit features are more informative for classification than the audio spectrogram features. To improve the weaker modality (audio), we could use a deeper CNN, better feature extraction, or multimodal fusion to combine audio and image information.

## 13 Problem 4: Multimodal Fusion (10 points)

Now you will play with multimodal fusion. Lets use a late fusion to improve our performance. We have provided some code with the hyperparameters to consider, but you are encouraged to play with them to try to get improvements. To make things simpler, the encoders for both modalities have been provided. However, some other parts are missing, so you will have to fill those in!

```
[ ]: import torch.nn as nn
from unimodals.common_models import MLP
from fusions.common_fusions import MultiplicativeInteractions2Modal, Concat
from training_structures.Supervised_Learning import train

# Image Encoder
class ImageEncoder(nn.Module):
    def __init__(self, output_dim=64):
        super().__init__()
        self.conv = nn.Sequential(
            nn.Conv2d(1, 32, 3, padding=1), nn.BatchNorm2d(32), nn.ReLU(),
            nn.MaxPool2d(2),
            nn.Conv2d(32, 64, 3, padding=1), nn.BatchNorm2d(64), nn.ReLU(),
            nn.MaxPool2d(2),
            nn.Flatten(),
            nn.Linear(64 * 7 * 7, output_dim)
        )
    def forward(self, x):
        return self.conv(x)

# Audio Encoder
class AudioEncoder(nn.Module):
    def __init__(self, output_dim=64):
        super().__init__()
        self.conv = nn.Sequential(
            nn.Conv2d(1, 32, 3, stride=2, padding=1), nn.BatchNorm2d(32), nn.
            ↪ReLU(), # 112 -> 56
            nn.MaxPool2d(2), # 56 -> 28
            nn.Conv2d(32, 64, 3, stride=2, padding=1), nn.BatchNorm2d(64), nn.
            ↪ReLU(), # 28 -> 14
            nn.Flatten(),
            nn.Linear(64 * 14 * 14, output_dim)
        )
    def forward(self, x):
        return self.conv(x)

# TODO: Encoders - create the list of encoders, images should be first, then
↪audio
encoders = [
    ImageEncoder(output_dim=64),
```

```

        AudioEncoder(output_dim=64)
    ]

    # TODO: Use the concat fusion as the fusion of choice
    fusion = Concat()

    # TODO: Create the head, which learns the joint features.
    # This should be an MLP that takes with input size based
    # on output size of your concationation, a hidden layer of size 256, and output_
    ↪ layer
    # of size 10.

    head = MLP(128, 256, 10).cuda()

    # Run Training
    print("Starting Training...")
    train(encoders, fusion, head, traindata, validdata, 5,
          task="classification", optmintype=torch.optim.AdamW, is_packed=False,
          lr=5e-4, save='avmnist_lmf.pt', weight_decay=0.001,
          objective=torch.nn.CrossEntropyLoss())

    # Run Test
    model = torch.load('avmnist_lmf.pt').cuda()
    test(model, testdata, 'avmnist', is_packed=False, task="classification",
          criterion=torch.nn.CrossEntropyLoss(), no_robust=True)

```

Starting Training..

Epoch 0 train loss: tensor(0.8997, device='cuda:0', grad\_fn=<DivBackward0>)

Epoch 0 valid loss: tensor(0.7256, device='cuda:0') acc: 0.7268

Saving Best

/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py:558:

UserWarning: This DataLoader will create 8 worker processes in total. Our suggested max number of worker in current system is 2, which is smaller than what this DataLoader is going to create. Please be aware that excessive worker creation might get DataLoader running slow or even freeze, lower the worker number to avoid potential slowness/freeze if necessary.

warnings.warn(\_create\_warning\_msg(

Epoch 1 train loss: tensor(0.7578, device='cuda:0', grad\_fn=<DivBackward0>)

Epoch 1 valid loss: tensor(0.7118, device='cuda:0') acc: 0.7378

Saving Best

/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py:558:

UserWarning: This DataLoader will create 8 worker processes in total. Our suggested max number of worker in current system is 2, which is smaller than what this DataLoader is going to create. Please be aware that excessive worker creation might get DataLoader running slow or even freeze, lower the worker number to avoid potential slowness/freeze if necessary.

```

warnings.warn(_create_warning_msg(
Epoch 2 train loss: tensor(0.7142, device='cuda:0', grad_fn=<DivBackward0>)
Epoch 2 valid loss: tensor(0.7329, device='cuda:0') acc: 0.7278

/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py:558:
UserWarning: This DataLoader will create 8 worker processes in total. Our
suggested max number of worker in current system is 2, which is smaller than
what this DataLoader is going to create. Please be aware that excessive worker
creation might get DataLoader running slow or even freeze, lower the worker
number to avoid potential slowness/freeze if necessary.
  warnings.warn(_create_warning_msg(
Epoch 3 train loss: tensor(0.6840, device='cuda:0', grad_fn=<DivBackward0>)
Epoch 3 valid loss: tensor(0.7252, device='cuda:0') acc: 0.7356

/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py:558:
UserWarning: This DataLoader will create 8 worker processes in total. Our
suggested max number of worker in current system is 2, which is smaller than
what this DataLoader is going to create. Please be aware that excessive worker
creation might get DataLoader running slow or even freeze, lower the worker
number to avoid potential slowness/freeze if necessary.
  warnings.warn(_create_warning_msg(
Epoch 4 train loss: tensor(0.6514, device='cuda:0', grad_fn=<DivBackward0>)
Epoch 4 valid loss: tensor(0.7425, device='cuda:0') acc: 0.7268
Training Time: 66.52783823013306
Training Peak Mem: 8609.90625
Training Params: 1077258

/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py:558:
UserWarning: This DataLoader will create 8 worker processes in total. Our
suggested max number of worker in current system is 2, which is smaller than
what this DataLoader is going to create. Please be aware that excessive worker
creation might get DataLoader running slow or even freeze, lower the worker
number to avoid potential slowness/freeze if necessary.
  warnings.warn(_create_warning_msg(
acc: 0.7048
Inference Time: 2.909508228302002
Inference Params: 1077258

```

Answer the following: 1. (2 points) Sometimes when training you may notice the model gets stuck in a range of loss and never seems to get it's loss down. What does this suggest? What are some ways you can fix this? 2. (2 points) What are some other fusion methods we could use that we could use? Would they lead to improvements compared to early fusion? 3. (6 points) Explain the difference between early fusion techniques and late fusion techniques. Be sure to discuss their benefits and tradeoffs.

Answers: 1. If the loss gets stuck and does not decrease, it may suggest issues such as a poor learning rate, insufficient model capacity, or optimization problems. Possible fixes include adjusting the learning rate, changing the optimizer, modifying the architecture, or adding better regularization



and normalization. 2. Instead of early fusion, other fusion methods include multiplicative interactions, attention-based fusion, and gating mechanisms. These methods can capture more complex relationships between modalities and may improve performance compared to simple early fusion. 3. - Early fusion combines modalities at the input or feature level before the model processes them. It allows the model to learn joint representations early but can be sensitive to noise and differences between modalities.

- Late fusion processes each modality separately with its own encoder and combines the representations at the end. It is easier to train and more robust when modalities differ, but it may miss fine-grained cross-modal interactions.

## 14 Problem 5: Other Fusion Techniques (30 points)

Now, we want you to try implementing some of these fusion techniques on your dataset! For this part, you will implement these fusion techniques: 1. Early fusion 2. Late fusion 3. TensorFusion 4. Low-Rank Tensor (LMF) Fusion

**You cannot just import and use the functions available in Multibench to do this. In addition, use einsum where applicable. TO RECIEVE FULL CREDIT, THE FUSIONS YOU IMPLEMENTATION MUST WORK WITH THE DATASET YOU CREATED FROM HOMEWORK 1. YOU WILL HAVE TO CREATE A SIMPLE MODEL FOR EACH FUSION TECHNIQUE TO PLAY WITH.**

**(5 points)** In your write up, report the best validation accuracies of your multimodal model (don't forget to include what hyperparameters you included) after training and any modifications that had to be done to your data or model to be able to train on it. In addition, talk about which technique you believe would be best for your dataset and why that is.

**Design the fusion classes with the modalities you are specifically working with in mind. The example we worked through above with MOSI was meant as a showcase of fusion in action - we do not require you to use text, video and audio as the modalities. Use whichever ones you are working with!**

The code below provided is to be filled in with the models you set up for each technique. For an example, the first fusion technique has been done for you.

**Answer Here:**

## 15 Load and Preprocess My dataset

```
[ ]: !pip install datasets==2.18.0
!pip install -q openai-whisper librosa soundfile
```

```
Collecting datasets==2.18.0
```

```
  Downloading datasets-2.18.0-py3-none-any.whl.metadata (20 kB)
```

```
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from datasets==2.18.0) (3.24.3)
```

```
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.12/dist-packages (from datasets==2.18.0) (2.0.2)
```

```
Requirement already satisfied: pyarrow>=12.0.0 in
```

```

/usr/local/lib/python3.12/dist-packages (from datasets==2.18.0) (23.0.1)
Collecting pyarrow-hotfix (from datasets==2.18.0)
  Downloading pyarrow_hotfix-0.7-py3-none-any.whl.metadata (3.6 kB)
Requirement already satisfied: dill<0.3.9,>=0.3.0 in
/usr/local/lib/python3.12/dist-packages (from datasets==2.18.0) (0.3.8)
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages
(from datasets==2.18.0) (2.2.2)
Requirement already satisfied: requests>=2.19.0 in
/usr/local/lib/python3.12/dist-packages (from datasets==2.18.0) (2.32.4)
Requirement already satisfied: tqdm>=4.62.1 in /usr/local/lib/python3.12/dist-
packages (from datasets==2.18.0) (4.67.3)
Requirement already satisfied: xxhash in /usr/local/lib/python3.12/dist-packages
(from datasets==2.18.0) (3.6.0)
Requirement already satisfied: multiprocessing in /usr/local/lib/python3.12/dist-
packages (from datasets==2.18.0) (0.70.16)
Collecting fsspec<=2024.2.0,>=2023.1.0 (from
fsspec[http]<=2024.2.0,>=2023.1.0->datasets==2.18.0)
  Downloading fsspec-2024.2.0-py3-none-any.whl.metadata (6.8 kB)
Requirement already satisfied: aiohttp in /usr/local/lib/python3.12/dist-
packages (from datasets==2.18.0) (3.13.3)
Requirement already satisfied: huggingface-hub>=0.19.4 in
/usr/local/lib/python3.12/dist-packages (from datasets==2.18.0) (1.5.0)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-
packages (from datasets==2.18.0) (26.0)
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.12/dist-
packages (from datasets==2.18.0) (6.0.3)
Requirement already satisfied: aiohappyeyeballs>=2.5.0 in
/usr/local/lib/python3.12/dist-packages (from aiohttp->datasets==2.18.0) (2.6.1)
Requirement already satisfied: aiosignal>=1.4.0 in
/usr/local/lib/python3.12/dist-packages (from aiohttp->datasets==2.18.0) (1.4.0)
Requirement already satisfied: attrs>=17.3.0 in /usr/local/lib/python3.12/dist-
packages (from aiohttp->datasets==2.18.0) (25.4.0)
Requirement already satisfied: frozenlist>=1.1.1 in
/usr/local/lib/python3.12/dist-packages (from aiohttp->datasets==2.18.0) (1.8.0)
Requirement already satisfied: multidict<7.0,>=4.5 in
/usr/local/lib/python3.12/dist-packages (from aiohttp->datasets==2.18.0) (6.7.1)
Requirement already satisfied: propcache>=0.2.0 in
/usr/local/lib/python3.12/dist-packages (from aiohttp->datasets==2.18.0) (0.4.1)
Requirement already satisfied: yarl<2.0,>=1.17.0 in
/usr/local/lib/python3.12/dist-packages (from aiohttp->datasets==2.18.0)
(1.22.0)
Requirement already satisfied: hf-xet<2.0.0,>=1.2.0 in
/usr/local/lib/python3.12/dist-packages (from huggingface-
hub>=0.19.4->datasets==2.18.0) (1.3.1)
Requirement already satisfied: httpx<1,>=0.23.0 in
/usr/local/lib/python3.12/dist-packages (from huggingface-
hub>=0.19.4->datasets==2.18.0) (0.28.1)
Requirement already satisfied: typer in /usr/local/lib/python3.12/dist-packages

```

(from huggingface-hub>=0.19.4->datasets==2.18.0) (0.24.1)  
Requirement already satisfied: typing-extensions>=4.1.0 in  
/usr/local/lib/python3.12/dist-packages (from huggingface-  
hub>=0.19.4->datasets==2.18.0) (4.15.0)  
Requirement already satisfied: charset\_normalizer<4,>=2 in  
/usr/local/lib/python3.12/dist-packages (from  
requests>=2.19.0->datasets==2.18.0) (3.4.4)  
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-  
packages (from requests>=2.19.0->datasets==2.18.0) (3.11)  
Requirement already satisfied: urllib3<3,>=1.21.1 in  
/usr/local/lib/python3.12/dist-packages (from  
requests>=2.19.0->datasets==2.18.0) (2.5.0)  
Requirement already satisfied: certifi>=2017.4.17 in  
/usr/local/lib/python3.12/dist-packages (from  
requests>=2.19.0->datasets==2.18.0) (2026.2.25)  
Requirement already satisfied: python-dateutil>=2.8.2 in  
/usr/local/lib/python3.12/dist-packages (from pandas->datasets==2.18.0)  
(2.9.0.post0)  
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-  
packages (from pandas->datasets==2.18.0) (2025.2)  
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-  
packages (from pandas->datasets==2.18.0) (2025.3)  
Requirement already satisfied: anyio in /usr/local/lib/python3.12/dist-packages  
(from httpx<1,>=0.23.0->huggingface-hub>=0.19.4->datasets==2.18.0) (4.12.1)  
Requirement already satisfied: httpcore==1.\* in /usr/local/lib/python3.12/dist-  
packages (from httpx<1,>=0.23.0->huggingface-hub>=0.19.4->datasets==2.18.0)  
(1.0.9)  
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.12/dist-  
packages (from httpcore==1.\*->httpx<1,>=0.23.0->huggingface-  
hub>=0.19.4->datasets==2.18.0) (0.16.0)  
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-  
packages (from python-dateutil>=2.8.2->pandas->datasets==2.18.0) (1.17.0)  
Requirement already satisfied: click>=8.2.1 in /usr/local/lib/python3.12/dist-  
packages (from typer->huggingface-hub>=0.19.4->datasets==2.18.0) (8.3.1)  
Requirement already satisfied: shellingham>=1.3.0 in  
/usr/local/lib/python3.12/dist-packages (from typer->huggingface-  
hub>=0.19.4->datasets==2.18.0) (1.5.4)  
Requirement already satisfied: rich>=12.3.0 in /usr/local/lib/python3.12/dist-  
packages (from typer->huggingface-hub>=0.19.4->datasets==2.18.0) (13.9.4)  
Requirement already satisfied: annotated-doc>=0.0.2 in  
/usr/local/lib/python3.12/dist-packages (from typer->huggingface-  
hub>=0.19.4->datasets==2.18.0) (0.0.4)  
Requirement already satisfied: markdown-it-py>=2.2.0 in  
/usr/local/lib/python3.12/dist-packages (from rich>=12.3.0->typer->huggingface-  
hub>=0.19.4->datasets==2.18.0) (4.0.0)  
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in  
/usr/local/lib/python3.12/dist-packages (from rich>=12.3.0->typer->huggingface-  
hub>=0.19.4->datasets==2.18.0) (2.19.2)

Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dist-packages (from markdown-it-py>=2.2.0->rich>=12.3.0->typer->huggingface-hub>=0.19.4->datasets==2.18.0) (0.1.2)

Downloading datasets-2.18.0-py3-none-any.whl (510 kB)

510.5/510.5 kB

36.9 MB/s eta 0:00:00

Downloading fsspec-2024.2.0-py3-none-any.whl (170 kB)

170.9/170.9 kB

21.3 MB/s eta 0:00:00

Downloading pyarrow\_hotfix-0.7-py3-none-any.whl (7.9 kB)

Installing collected packages: pyarrow-hotfix, fsspec, datasets

Attempting uninstall: fsspec

Found existing installation: fsspec 2025.3.0

Uninstalling fsspec-2025.3.0:

Successfully uninstalled fsspec-2025.3.0

Attempting uninstall: datasets

Found existing installation: datasets 4.6.1

Uninstalling datasets-4.6.1:

Successfully uninstalled datasets-4.6.1

ERROR: pip's dependency resolver does not currently take into account all the packages that are installed. This behaviour is the source of the following dependency conflicts.

torchaudio 2.3.1 requires torch==2.3.1, but you have torch 2.10.0 which is incompatible.

gcsfs 2025.3.0 requires fsspec==2025.3.0, but you have fsspec 2024.2.0 which is incompatible.

Successfully installed datasets-2.18.0 fsspec-2024.2.0 pyarrow-hotfix-0.7

Traceback (most recent call last):

File "/usr/local/lib/python3.12/dist-packages/pip/\_internal/cli/base\_command.py", line 179, in exc\_logging\_wrapper  
status = run\_func(\*args)

File "/usr/local/lib/python3.12/dist-packages/pip/\_internal/cli/req\_command.py", line 67, in wrapper  
return func(self, options, args)

File "/usr/local/lib/python3.12/dist-packages/pip/\_internal/commands/install.py", line 447, in run  
~C

```
[ ]: from datasets import load_dataset  
  
dataset = load_dataset("superb", "er")
```

```
label_names = dataset["session1"].features["label"].names
print(label_names)
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94:
UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab
(https://huggingface.co/settings/tokens), set it as secret in your Google Colab
and restart your session.
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access
public models or datasets.
  warnings.warn(

Downloading readme: 0.00B [00:00, ?B/s]

['neu', 'hap', 'ang', 'sad']
```

```
[ ]: from datasets import concatenate_datasets, DatasetDict

train = concatenate_datasets([dataset["session1"], dataset["session2"],
    ↪dataset["session3"]])
val   = dataset["session4"]
test  = dataset["session5"]

ds = DatasetDict({
    "train": train,
    "validation": val,
    "test": test
})
```

```
[ ]: import whisper
import numpy as np

model = whisper.load_model("tiny")

def audio_to_text(audio, sr):
    audio = np.asarray(audio, dtype=np.float32)

    result = model.transcribe(
        audio,
        fp16=False,
        language="en",
        task="transcribe",
        verbose=False
    )
```

```
return result["text"].strip()
```

```
[ ]: import os
import pandas as pd
from tqdm import tqdm

OUT_ROOT = "/content/superb_er_preproc"
TRANSCRIPT_DIR = os.path.join(OUT_ROOT, "transcripts")

os.makedirs(TRANSCRIPT_DIR, exist_ok=True)

MAX_PER_SPLIT = 50 # debug

def process_split(split_name, split_ds):

    rows = []

    for i in tqdm(range(min(len(split_ds), MAX_PER_SPLIT)), desc=split_name):

        ex = split_ds[i]

        audio = ex["audio"]["array"]
        sr = ex["audio"]["sampling_rate"]

        text = audio_to_text(audio, sr)

        ex_id = f"{split_name}_{i:06d}"

        txt_path = os.path.join(TRANSCRIPT_DIR, f"{ex_id}.txt")

        with open(txt_path, "w") as f:
            f.write(text)

        rows.append({
            "id": ex_id,
            "split": split_name,
            "label": label_names[ex["label"]],
            "transcript": text,
            "transcript_path": txt_path
        })

    return pd.DataFrame(rows)
```

```
[ ]: text_meta = pd.concat([
    process_split("train", ds["train"]),
    process_split("validation", ds["validation"]),
```

```

    process_split("test", ds["test"])
])

os.makedirs(OUT_ROOT, exist_ok=True)

meta_path = os.path.join(OUT_ROOT, "metadata_with_text.csv")

text_meta.to_csv(meta_path, index=False)

print("Saved:", meta_path)
text_meta.head()

```

```

train:  0%|          | 0/50 [00:00<?, ?it/s]
  0%|          | 0/194 [00:00<?, ?frames/s]
100%|      | 194/194 [00:00<00:00, 309.47frames/s]
train:  2%|          | 1/50 [00:03<02:29, 3.04s/it]
100%|      | 138/138 [00:00<00:00, 2220.69frames/s]

  0%|          | 0/313 [00:00<?, ?frames/s]
100%|      | 313/313 [00:00<00:00, 2196.69frames/s]
train:  6%|          | 3/50 [00:03<00:41, 1.12it/s]
  0%|          | 0/402 [00:00<?, ?frames/s]
100%|      | 402/402 [00:00<00:00, 2602.29frames/s]
train:  8%|          | 4/50 [00:03<00:29, 1.53it/s]
100%|      | 275/275 [00:00<00:00, 2963.88frames/s]
train: 10%|          | 5/50 [00:03<00:21, 2.10it/s]
  0%|          | 0/507 [00:00<?, ?frames/s]
100%|      | 507/507 [00:00<00:00, 3276.54frames/s]
train: 12%|          | 6/50 [00:03<00:16, 2.61it/s]
  0%|          | 0/723 [00:00<?, ?frames/s]
100%|      | 723/723 [00:00<00:00, 3665.68frames/s]
train: 14%|          | 7/50 [00:04<00:14, 3.01it/s]
  0%|          | 0/602 [00:00<?, ?frames/s]
100%|      | 602/602 [00:00<00:00, 2876.38frames/s]
train: 16%|          | 8/50 [00:04<00:12, 3.31it/s]
  0%|          | 0/933 [00:00<?, ?frames/s]
100%|      | 933/933 [00:00<00:00, 1477.67frames/s]
train: 18%|          | 9/50 [00:04<00:16, 2.44it/s]
100%|      | 293/293 [00:00<00:00, 6832.56frames/s]

100%|      | 572/572 [00:00<00:00, 6731.49frames/s]
train: 22%|          | 11/50 [00:05<00:10, 3.79it/s]
100%|      | 565/565 [00:00<00:00, 6960.12frames/s]
train: 24%|          | 12/50 [00:05<00:08, 4.43it/s]
100%|      | 576/576 [00:00<00:00, 6752.24frames/s]
train: 26%|          | 13/50 [00:05<00:07, 5.09it/s]
100%|      | 256/256 [00:00<00:00, 6694.86frames/s]

```

```

100%|      | 421/421 [00:00<00:00, 5818.91frames/s]
train: 30%|      | 15/50 [00:05<00:05, 6.73it/s]
100%|      | 388/388 [00:00<00:00, 6798.30frames/s]

100%|      | 425/425 [00:00<00:00, 7978.64frames/s]
train: 34%|      | 17/50 [00:05<00:04, 8.06it/s]
100%|      | 303/303 [00:00<00:00, 5198.78frames/s]

100%|      | 612/612 [00:00<00:00, 10235.23frames/s]
train: 38%|      | 19/50 [00:05<00:03, 8.97it/s]
100%|      | 740/740 [00:00<00:00, 9768.84frames/s]

100%|      | 358/358 [00:00<00:00, 6699.09frames/s]
train: 42%|      | 21/50 [00:06<00:03, 9.39it/s]
100%|      | 218/218 [00:00<00:00, 4263.48frames/s]

0%|      | 0/665 [00:00<?, ?frames/s]
100%|      | 665/665 [00:00<00:00, 5278.37frames/s]
train: 46%|      | 23/50 [00:06<00:03, 8.92it/s]
100%|      | 305/305 [00:00<00:00, 5399.03frames/s]

100%|      | 305/305 [00:00<00:00, 5005.51frames/s]
train: 50%|      | 25/50 [00:06<00:02, 9.61it/s]
100%|      | 452/452 [00:00<00:00, 6875.56frames/s]

0%|      | 0/579 [00:00<?, ?frames/s]
100%|      | 579/579 [00:00<00:00, 5151.02frames/s]
train: 54%|      | 27/50 [00:06<00:02, 9.19it/s]
0%|      | 0/494 [00:00<?, ?frames/s]
100%|      | 494/494 [00:00<00:00, 3889.04frames/s]
train: 56%|      | 28/50 [00:06<00:02, 8.48it/s]
100%|      | 367/367 [00:00<00:00, 4676.61frames/s]
train: 58%|      | 29/50 [00:06<00:02, 8.60it/s]
100%|      | 421/421 [00:00<00:00, 7552.00frames/s]

100%|      | 321/321 [00:00<00:00, 9055.80frames/s]
train: 62%|      | 31/50 [00:07<00:01, 9.72it/s]
100%|      | 468/468 [00:00<00:00, 5985.01frames/s]
train: 64%|      | 32/50 [00:07<00:01, 9.56it/s]
100%|      | 473/473 [00:00<00:00, 5698.85frames/s]
train: 66%|      | 33/50 [00:07<00:01, 9.38it/s]
100%|      | 461/461 [00:00<00:00, 7044.35frames/s]

100%|      | 125/125 [00:00<00:00, 3629.47frames/s]
train: 70%|      | 35/50 [00:07<00:01, 10.43it/s]
100%|      | 286/286 [00:00<00:00, 5051.25frames/s]

```



```

100%|      | 343/343 [00:00<00:00, 6883.15frames/s]
train: 74%|      | 37/50 [00:07<00:01, 11.05it/s]
100%|      | 256/256 [00:00<00:00, 4232.50frames/s]

100%|      | 376/376 [00:00<00:00, 6558.53frames/s]
train: 78%|      | 39/50 [00:07<00:00, 11.10it/s]
100%|      | 258/258 [00:00<00:00, 4358.09frames/s]

  0%|      | 0/634 [00:00<?, ?frames/s]
100%|      | 634/634 [00:00<00:00, 5873.09frames/s]
train: 82%|      | 41/50 [00:08<00:00, 10.20it/s]
100%|      | 267/267 [00:00<00:00, 2688.15frames/s]

100%|      | 169/169 [00:00<00:00, 2478.45frames/s]
train: 86%|      | 43/50 [00:08<00:00, 9.72it/s]
100%|      | 320/320 [00:00<00:00, 4705.60frames/s]

  0%|      | 0/228 [00:00<?, ?frames/s]
100%|      | 228/228 [00:00<00:00, 725.88frames/s]
train: 90%|      | 45/50 [00:08<00:00, 7.10it/s]
100%|      | 512/512 [00:00<00:00, 7180.75frames/s]
train: 92%|      | 46/50 [00:08<00:00, 7.45it/s]
100%|      | 457/457 [00:00<00:00, 6610.12frames/s]

100%|      | 205/205 [00:00<00:00, 3386.45frames/s]
train: 96%|      | 48/50 [00:09<00:00, 8.26it/s]
100%|      | 298/298 [00:00<00:00, 5945.18frames/s]

  0%|      | 0/292 [00:00<?, ?frames/s]
100%|      | 292/292 [00:00<00:00, 481.08frames/s]
train: 100%|      | 50/50 [00:09<00:00, 5.10it/s]
validation: 0%|      | 0/50 [00:00<?, ?it/s]
100%|      | 300/300 [00:00<00:00, 3836.43frames/s]
validation: 2%|      | 1/50 [00:00<00:05, 9.26it/s]
  0%|      | 0/410 [00:00<?, ?frames/s]
100%|      | 410/410 [00:00<00:00, 3875.96frames/s]
validation: 4%|      | 2/50 [00:00<00:06, 7.65it/s]
  0%|      | 0/701 [00:00<?, ?frames/s]
100%|      | 701/701 [00:00<00:00, 5959.28frames/s]
validation: 6%|      | 3/50 [00:00<00:06, 7.17it/s]
  0%|      | 0/677 [00:00<?, ?frames/s]
100%|      | 677/677 [00:00<00:00, 4236.75frames/s]
validation: 8%|      | 4/50 [00:00<00:07, 5.99it/s]
  0%|      | 0/577 [00:00<?, ?frames/s]
100%|      | 577/577 [00:00<00:00, 4581.21frames/s]
validation: 10%|      | 5/50 [00:00<00:07, 6.05it/s]
  0%|      | 0/1002 [00:00<?, ?frames/s]
100%|      | 1002/1002 [00:00<00:00, 3194.73frames/s]

```

```

validation: 12%|          | 6/50 [00:01<00:10, 4.37it/s]
0%|          | 0/444 [00:00<?, ?frames/s]
100%|        | 444/444 [00:00<00:00, 3223.29frames/s]
validation: 14%|          | 7/50 [00:01<00:08, 4.82it/s]
100%|        | 203/203 [00:00<00:00, 3845.47frames/s]

0%|          | 0/360 [00:00<?, ?frames/s]
100%|        | 360/360 [00:00<00:00, 2579.65frames/s]
validation: 18%|          | 9/50 [00:01<00:06, 6.12it/s]
0%|          | 0/697 [00:00<?, ?frames/s]
100%|        | 697/697 [00:00<00:00, 4505.70frames/s]
validation: 20%|          | 10/50 [00:01<00:06, 5.99it/s]
100%|        | 171/171 [00:00<00:00, 2802.80frames/s]

100%|        | 340/340 [00:00<00:00, 3583.60frames/s]
validation: 24%|          | 12/50 [00:01<00:05, 7.26it/s]
100%|        | 256/256 [00:00<00:00, 4119.19frames/s]

0%|          | 0/498 [00:00<?, ?frames/s]
100%|        | 498/498 [00:00<00:00, 4107.98frames/s]
validation: 28%|          | 14/50 [00:02<00:04, 7.82it/s]
100%|        | 272/272 [00:00<00:00, 5226.40frames/s]

100%|        | 251/251 [00:00<00:00, 4462.61frames/s]
validation: 32%|          | 16/50 [00:02<00:03, 9.19it/s]
0%|          | 0/425 [00:00<?, ?frames/s]
100%|        | 425/425 [00:00<00:00, 4084.55frames/s]
validation: 34%|          | 17/50 [00:02<00:03, 8.85it/s]
100%|        | 361/361 [00:00<00:00, 6352.87frames/s]

100%|        | 275/275 [00:00<00:00, 5562.74frames/s]
validation: 38%|          | 19/50 [00:02<00:03, 10.08it/s]
0%|          | 0/1023 [00:00<?, ?frames/s]
100%|        | 1023/1023 [00:00<00:00, 6677.31frames/s]

0%|          | 0/547 [00:00<?, ?frames/s]
100%|        | 547/547 [00:00<00:00, 4486.31frames/s]
validation: 42%|          | 21/50 [00:02<00:03, 8.23it/s]
0%|          | 0/397 [00:00<?, ?frames/s]
100%|        | 397/397 [00:00<00:00, 3640.34frames/s]
validation: 44%|          | 22/50 [00:03<00:03, 7.92it/s]
100%|        | 312/312 [00:00<00:00, 4791.17frames/s]

0%|          | 0/729 [00:00<?, ?frames/s]
100%|        | 729/729 [00:00<00:00, 4780.46frames/s]
validation: 48%|          | 24/50 [00:03<00:03, 7.80it/s]
100%|        | 952/952 [00:00<00:00, 10535.31frames/s]
validation: 50%|          | 25/50 [00:03<00:03, 7.95it/s]

```

```

100%|      | 256/256 [00:00<00:00, 4424.33frames/s]

0%|        | 0/496 [00:00<?, ?frames/s]
100%|      | 496/496 [00:00<00:00, 3173.94frames/s]
validation: 54%|      | 27/50 [00:03<00:02, 7.90it/s]
100%|      | 269/269 [00:00<00:00, 5737.82frames/s]

100%|      | 237/237 [00:00<00:00, 4604.11frames/s]
validation: 58%|      | 29/50 [00:03<00:02, 9.41it/s]
100%|      | 264/264 [00:00<00:00, 4036.00frames/s]

0%|        | 0/950 [00:00<?, ?frames/s]
100%|      | 950/950 [00:00<00:00, 3639.62frames/s]
validation: 62%|      | 31/50 [00:04<00:02, 7.59it/s]
100%|      | 152/152 [00:00<00:00, 3284.02frames/s]

100%|      | 428/428 [00:00<00:00, 6767.28frames/s]
validation: 66%|      | 33/50 [00:04<00:01, 8.65it/s]
100%|      | 411/411 [00:00<00:00, 5504.74frames/s]

0%|        | 0/865 [00:00<?, ?frames/s]
100%|      | 865/865 [00:00<00:00, 5407.76frames/s]
validation: 70%|      | 35/50 [00:04<00:01, 8.01it/s]
0%|        | 0/693 [00:00<?, ?frames/s]
100%|      | 693/693 [00:00<00:00, 4899.89frames/s]
validation: 72%|      | 36/50 [00:04<00:01, 7.43it/s]
100%|      | 292/292 [00:00<00:00, 3147.00frames/s]
validation: 74%|      | 37/50 [00:04<00:01, 7.64it/s]
0%|        | 0/881 [00:00<?, ?frames/s]
100%|      | 881/881 [00:00<00:00, 6361.91frames/s]
validation: 76%|      | 38/50 [00:05<00:01, 7.14it/s]
0%|        | 0/670 [00:00<?, ?frames/s]
100%|      | 670/670 [00:00<00:00, 3615.59frames/s]
validation: 78%|      | 39/50 [00:05<00:01, 6.29it/s]
0%|        | 0/683 [00:00<?, ?frames/s]
100%|      | 683/683 [00:00<00:00, 3836.28frames/s]
validation: 80%|      | 40/50 [00:05<00:01, 5.85it/s]
0%|        | 0/379 [00:00<?, ?frames/s]
100%|      | 379/379 [00:00<00:00, 3073.71frames/s]
validation: 82%|      | 41/50 [00:05<00:01, 6.05it/s]
0%|        | 0/973 [00:00<?, ?frames/s]
100%|      | 973/973 [00:00<00:00, 3923.05frames/s]
validation: 84%|      | 42/50 [00:05<00:01, 4.99it/s]
100%|      | 290/290 [00:00<00:00, 4477.30frames/s]

100%|      | 418/418 [00:00<00:00, 4380.27frames/s]
validation: 88%|      | 44/50 [00:06<00:00, 6.23it/s]
100%|      | 151/151 [00:00<00:00, 2765.75frames/s]

```

```

100%|      | 352/352 [00:00<00:00, 3941.64frames/s]
validation: 92%|      | 46/50 [00:06<00:00, 7.36it/s]
100%|      | 180/180 [00:00<00:00, 4153.07frames/s]

100%|      | 163/163 [00:00<00:00, 2362.17frames/s]
validation: 96%|      | 48/50 [00:06<00:00, 8.65it/s]
100%|      | 240/240 [00:00<00:00, 4151.83frames/s]

0%|        | 0/1093 [00:00<?, ?frames/s]
100%|      | 1093/1093 [00:00<00:00, 3832.11frames/s]
validation: 100%|      | 50/50 [00:06<00:00, 7.22it/s]
test: 0%|      | 0/50 [00:00<?, ?it/s]
100%|      | 255/255 [00:00<00:00, 3630.84frames/s]

0%|        | 0/423 [00:00<?, ?frames/s]
100%|      | 423/423 [00:00<00:00, 2831.58frames/s]
test: 4%|      | 2/50 [00:00<00:06, 7.36it/s]
0%|        | 0/601 [00:00<?, ?frames/s]
100%|      | 601/601 [00:00<00:00, 2597.83frames/s]
test: 6%|      | 3/50 [00:00<00:08, 5.30it/s]
100%|      | 156/156 [00:00<00:00, 3064.35frames/s]

0%|        | 0/481 [00:00<?, ?frames/s]
100%|      | 481/481 [00:00<00:00, 2813.70frames/s]
test: 10%|     | 5/50 [00:00<00:07, 6.11it/s]
0%|        | 0/900 [00:00<?, ?frames/s]
100%|      | 900/900 [00:00<00:00, 2807.68frames/s]
test: 12%|     | 6/50 [00:01<00:09, 4.58it/s]
0%|        | 0/676 [00:00<?, ?frames/s]
100%|      | 676/676 [00:00<00:00, 3446.49frames/s]
test: 14%|     | 7/50 [00:01<00:09, 4.57it/s]
0%|        | 0/218 [00:00<?, ?frames/s]
100%|      | 218/218 [00:00<00:00, 740.48frames/s]
test: 16%|     | 8/50 [00:01<00:10, 4.06it/s]
100%|      | 566/566 [00:00<00:00, 5962.35frames/s]
test: 18%|     | 9/50 [00:01<00:08, 4.76it/s]
100%|      | 230/230 [00:00<00:00, 4248.50frames/s]

100%|      | 349/349 [00:00<00:00, 4078.74frames/s]
test: 22%|     | 11/50 [00:02<00:06, 6.42it/s]
100%|      | 268/268 [00:00<00:00, 5705.32frames/s]

100%|      | 415/415 [00:00<00:00, 6711.56frames/s]
test: 26%|     | 13/50 [00:02<00:04, 8.04it/s]
0%|        | 0/1095 [00:00<?, ?frames/s]
100%|      | 1095/1095 [00:00<00:00, 5169.91frames/s]
test: 28%|     | 14/50 [00:02<00:05, 6.70it/s]

```

```

100%|      | 225/225 [00:00<00:00, 5718.92frames/s]

100%|      | 497/497 [00:00<00:00, 7542.76frames/s]
test: 32%|      | 16/50 [00:02<00:04, 8.30it/s]
0%|      | 0/314 [00:00<?, ?frames/s]
100%|      | 314/314 [00:00<00:00, 2537.30frames/s]
test: 34%|      | 17/50 [00:02<00:04, 7.92it/s]
100%|      | 587/587 [00:00<00:00, 8091.76frames/s]

100%|      | 321/321 [00:00<00:00, 5764.34frames/s]
test: 38%|      | 19/50 [00:02<00:03, 9.03it/s]
0%|      | 0/175 [00:00<?, ?frames/s]
100%|      | 175/175 [00:00<00:00, 1480.20frames/s]
test: 40%|      | 20/50 [00:03<00:03, 8.60it/s]
100%|      | 185/185 [00:00<00:00, 2342.27frames/s]
test: 42%|      | 21/50 [00:03<00:03, 8.81it/s]
0%|      | 0/1047 [00:00<?, ?frames/s]
100%|      | 1047/1047 [00:00<00:00, 6556.43frames/s]
test: 44%|      | 22/50 [00:03<00:03, 7.56it/s]
100%|      | 337/337 [00:00<00:00, 3785.92frames/s]
test: 46%|      | 23/50 [00:03<00:03, 7.87it/s]
100%|      | 387/387 [00:00<00:00, 5233.92frames/s]
test: 48%|      | 24/50 [00:03<00:03, 8.35it/s]
100%|      | 267/267 [00:00<00:00, 4146.69frames/s]

100%|      | 420/420 [00:00<00:00, 10661.94frames/s]
test: 52%|      | 26/50 [00:03<00:02, 10.07it/s]
100%|      | 174/174 [00:00<00:00, 4849.26frames/s]

100%|      | 120/120 [00:00<00:00, 3424.64frames/s]
test: 56%|      | 28/50 [00:03<00:01, 12.15it/s]
100%|      | 813/813 [00:00<00:00, 9094.03frames/s]

100%|      | 282/282 [00:00<00:00, 5997.70frames/s]
test: 60%|      | 30/50 [00:03<00:01, 11.72it/s]
100%|      | 218/218 [00:00<00:00, 4426.15frames/s]

0%|      | 0/307 [00:00<?, ?frames/s]
100%|      | 307/307 [00:00<00:00, 2776.20frames/s]
test: 64%|      | 32/50 [00:04<00:01, 11.06it/s]
100%|      | 298/298 [00:00<00:00, 4301.01frames/s]

100%|      | 421/421 [00:00<00:00, 4336.68frames/s]
test: 68%|      | 34/50 [00:04<00:01, 10.49it/s]
0%|      | 0/746 [00:00<?, ?frames/s]
100%|      | 746/746 [00:00<00:00, 5982.02frames/s]

100%|      | 346/346 [00:00<00:00, 3967.29frames/s]

```

```

test: 72%|          | 36/50 [00:04<00:01, 9.35it/s]
100%|          | 198/198 [00:00<00:00, 4076.54frames/s]

0%|          | 0/1087 [00:00<?, ?frames/s]
100%|          | 1087/1087 [00:00<00:00, 6523.76frames/s]
test: 76%|          | 38/50 [00:04<00:01, 8.76it/s]
0%|          | 0/681 [00:00<?, ?frames/s]
100%|          | 681/681 [00:00<00:00, 4812.34frames/s]
test: 78%|          | 39/50 [00:05<00:01, 8.08it/s]
0%|          | 0/209 [00:00<?, ?frames/s]
100%|          | 209/209 [00:00<00:00, 908.18frames/s]
test: 80%|          | 40/50 [00:05<00:01, 6.65it/s]
100%|          | 183/183 [00:00<00:00, 3969.39frames/s]

0%|          | 0/933 [00:00<?, ?frames/s]
100%|          | 933/933 [00:00<00:00, 8221.34frames/s]
test: 84%|          | 42/50 [00:05<00:01, 7.55it/s]
0%|          | 0/625 [00:00<?, ?frames/s]
100%|          | 625/625 [00:00<00:00, 4667.34frames/s]
test: 86%|          | 43/50 [00:05<00:00, 7.25it/s]
0%|          | 0/212 [00:00<?, ?frames/s]
100%|          | 212/212 [00:00<00:00, 853.52frames/s]
test: 88%|          | 44/50 [00:05<00:01, 5.90it/s]
0%|          | 0/657 [00:00<?, ?frames/s]
100%|          | 657/657 [00:00<00:00, 3380.25frames/s]
test: 90%|          | 45/50 [00:06<00:00, 5.49it/s]
100%|          | 231/231 [00:00<00:00, 6100.59frames/s]

100%|          | 173/173 [00:00<00:00, 3245.60frames/s]
test: 94%|          | 47/50 [00:06<00:00, 7.44it/s]
100%|          | 187/187 [00:00<00:00, 3779.98frames/s]

100%|          | 112/112 [00:00<00:00, 3324.57frames/s]
test: 98%|          | 49/50 [00:06<00:00, 9.37it/s]
0%|          | 0/113 [00:00<?, ?frames/s]
100%|          | 113/113 [00:00<00:00, 518.94frames/s]
test: 100%|          | 50/50 [00:06<00:00, 7.51it/s]

Saved: /content/superb_er_preproc/metadata_with_text.csv

```

| [ ]: | id           | split | label | transcript \                                |
|------|--------------|-------|-------|---------------------------------------------|
| 0    | train_000000 | train | neu   | Excuse me?                                  |
| 1    | train_000001 | train | neu   | Yeah.                                       |
| 2    | train_000002 | train | neu   | Is there a problem?                         |
| 3    | train_000003 | train | neu   | Well, what's the problem? Let me change it. |
| 4    | train_000004 | train | ang   | That's out of control.                      |

```

                                transcript_path
0 /content/superb_er_preproc/transcripts/train_0...
1 /content/superb_er_preproc/transcripts/train_0...
2 /content/superb_er_preproc/transcripts/train_0...
3 /content/superb_er_preproc/transcripts/train_0...
4 /content/superb_er_preproc/transcripts/train_0...

```

## 16 Model

### 16.0.1 Dataset loader

```

[ ]: import torch
from torch.utils.data import Dataset
import librosa
import numpy as np

class MultimodalDataset(Dataset):

    def __init__(self, hf_dataset, max_len=128, text_len=32):
        self.ds = hf_dataset
        self.max_len = max_len
        self.text_len = text_len

    def __len__(self):
        return len(self.ds)

    def text_to_tensor(self, text):

        ids = [ord(c) % 30522 for c in text][:self.text_len]

        if len(ids) < self.text_len:
            ids += [0]*(self.text_len-len(ids))

        return torch.tensor(ids)

    def __getitem__(self, idx):

        ex = self.ds[idx]

        # AUDIO
        audio = np.asarray(ex["audio"]["array"], dtype=np.float32)
        sr = ex["audio"]["sampling_rate"]

        mel = librosa.feature.melspectrogram(
            y=audio,
            sr=sr,

```

```

        n_mels=64
    )

    MAX_LEN = self.max_len
    mel = mel[:, :MAX_LEN]

    if mel.shape[1] < MAX_LEN:
        mel = np.pad(mel, ((0,0),(0,MAX_LEN-mel.shape[1])))

    mel = torch.tensor(mel).float()

    # TEXT
    text = ""
    text_tensor = self.text_to_tensor(text)

    # LABEL
    label = torch.tensor(ex["label"])

    return mel, text_tensor, label

```

## 16.0.2 Encoders

```

[ ]: import torch.nn as nn

class AudioEncoder(nn.Module):

    def __init__(self):
        super().__init__()

        self.net = nn.Sequential(
            nn.Conv2d(1,16,3,padding=1),
            nn.ReLU(),
            nn.AdaptiveAvgPool2d((8,8)),
            nn.Flatten(),
            nn.Linear(16*8*8,64)
        )

    def forward(self,x):
        x = x.unsqueeze(1)
        return self.net(x)

```

```

[ ]: class TextEncoder(nn.Module):

    def __init__(self, vocab=30522):
        super().__init__()

        self.embed = nn.Embedding(vocab, 128)

```



```

        self.pool = nn.AdaptiveAvgPool1d(1)
        self.fc = nn.Linear(128,64)

    def forward(self,x):

        x = self.embed(x)
        x = x.mean(dim=1)
        return self.fc(x)

```

## 17 Fusion

```

[ ]: # Imports incase you need them again! Feel free to include anything else you
    ↪ need
import torch
from torch import nn
from torch.nn import functional as F
import pdb
from torch.autograd import Variable

```

## 18 Early Fusion

```

[ ]: class EarlyFusion(nn.Module):

    def forward(self, x):

        return torch.cat((x[0], x[1]), dim=1)

```

## 19 (5 Points) Late Fusion

```

[ ]: class LateFusion(nn.Module):
    def __init__(self):
        super(LateFusion, self).__init__()

    def forward(self, x):
        return torch.cat((x[0], x[1]), dim=1)

```

## 20 (5 points) Tensor Fusion

```

[ ]: class TensorFusion(nn.Module):

    def __init__(self):
        super().__init__()

```

```

        self.proj = nn.Linear(64*64,128)

    def forward(self,x):

        fusion = torch.einsum('bi,bj->bij', x[0], x[1])
        fusion = fusion.reshape(fusion.size(0), -1)

        fusion = self.proj(fusion)

        return fusion

```

## 21 (5 Points) Low-Rank Tensor Fusion (LMF) Fusion

```

[ ]: class LMFFusion(nn.Module):

    def __init__(self, rank=4, dim=64, output_dim=128):
        super().__init__()

        self.rank = rank

        self.audio_factor = nn.Parameter(torch.randn(rank, dim))
        self.text_factor = nn.Parameter(torch.randn(rank, dim))

        self.proj = nn.Linear(rank, output_dim)

    def forward(self, x):

        a, b = x

        a = torch.einsum("bi,ri->br", a, self.audio_factor)
        b = torch.einsum("bi,ri->br", b, self.text_factor)

        fusion = a * b

        return self.proj(fusion)

```

## 22 Multimodal model & Training loop

```

[ ]: class MultimodalModel(nn.Module):

    def __init__(self,fusion):

        super().__init__()

        self.audio = AudioEncoder()

```

```

        self.text = TextEncoder()

        self.fusion = fusion

        self.classifier = nn.Linear(128,4)

    def forward(self, audio, text):

        a = self.audio(audio)
        t = self.text(text)

        fused = self.fusion([a,t])

        return self.classifier(fused)

```

```

[ ]: from torch.utils.data import DataLoader

train_dataset = MultimodalDataset(ds["train"])

loader = DataLoader(train_dataset, batch_size=16, shuffle=True)

optimizer = torch.optim.Adam(model.parameters(), lr=5e-4)

criterion = nn.CrossEntropyLoss()

```

```

[ ]: import time
def exp(fusion):
    start = time.time()

    fusion_name = fusion.__class__.__name__
    print(f"\n===== Running {fusion_name} =====")

    model = MultimodalModel(fusion).cuda()
    optimizer = torch.optim.Adam(model.parameters(), lr=5e-4)

    best_acc = 0

    for epoch in range(5):

        model.train()

        total_loss = 0
        correct = 0
        total = 0

        for audio, text, label in loader:

```

```

        audio = audio.cuda()
        text = text.cuda()
        label = label.cuda()

        pred = model(audio, text)

        loss = criterion(pred, label)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        total_loss += loss.item()

        preds = pred.argmax(dim=1)
        correct += (preds == label).sum().item()
        total += label.size(0)

    avg_loss = total_loss / len(loader)
    acc = correct / total

    best_acc = max(best_acc, acc)

    print(f"Epoch {epoch} | Loss {avg_loss:.4f} | Acc {acc:.4f}")

train_time = time.time() - start

print("Training time:", train_time)
print(f"Best Accuracy ({fusion_name}): {best_acc:.4f}")

```

```

[ ]: exp(EarlyFusion())
     exp(LateFusion())

```

```

===== Running EarlyFusion =====
Epoch 0 | Loss 1.3234 | Acc 0.4060
Epoch 1 | Loss 1.2517 | Acc 0.4204
Epoch 2 | Loss 1.2190 | Acc 0.4250
Epoch 3 | Loss 1.1851 | Acc 0.4461
Epoch 4 | Loss 1.1788 | Acc 0.4452
Best Accuracy (EarlyFusion): 0.4461

```

```

===== Running LateFusion =====
Epoch 0 | Loss 1.3434 | Acc 0.3897
Epoch 1 | Loss 1.2474 | Acc 0.3992
Epoch 2 | Loss 1.2135 | Acc 0.4204
Epoch 3 | Loss 1.1946 | Acc 0.4400

```

```
Epoch 4 | Loss 1.1772 | Acc 0.4566  
Best Accuracy (LateFusion): 0.4566
```

```
[ ]: 0.45658177355016877
```

```
[ ]: exp(TensorFusion())
```

```
===== Running TensorFusion =====  
Epoch 0 | Loss 1.3895 | Acc 0.3743  
Epoch 1 | Loss 1.2915 | Acc 0.4007  
Epoch 2 | Loss 1.2541 | Acc 0.4053  
Epoch 3 | Loss 1.2105 | Acc 0.4268  
Epoch 4 | Loss 1.1964 | Acc 0.4486  
Best Accuracy (TensorFusion): 0.4486
```

```
[ ]: 0.44860386621663084
```

```
[ ]: exp(LMFFusion())
```

```
===== Running LMFFusion =====  
Epoch 0 | Loss 1.7826 | Acc 0.3793  
Epoch 1 | Loss 1.3044 | Acc 0.3946  
Epoch 2 | Loss 1.4334 | Acc 0.3921  
Epoch 3 | Loss 2.8619 | Acc 0.3559  
Epoch 4 | Loss 1.2810 | Acc 0.3925  
Best Accuracy (LMFFusion): 0.3946
```

```
[ ]: 0.39459957042037436
```

## 23 Results

We trained four fusion models (Early, Late, Tensor, and LMF) for 5 epochs using Adam ( $\text{lr} = 5\text{e-}4$ ) with batch size = 16. The encoders produced 64-dimensional audio and text features, and the classifier used a 128-dim hidden representation for most fusion methods. To make training stable, audio was converted to 64-mel spectrograms and padded/truncated to length 128, and text was converted to fixed-length token tensors (length 32).

The best accuracies obtained were:

Early Fusion: 0.446

Late Fusion: 0.457

Tensor Fusion: 0.449

LMF Fusion: 0.395

Late fusion achieved the best performance. This likely occurs because each modality (audio and text) can first learn its own representation before being combined, which leads to more stable features. Tensor fusion can capture cross-modal interactions but introduces a large feature space that is harder to train with limited data. LMF performed worse, likely due to the low rank approximation limiting representational capacity. Overall, Late Fusion appears most suitable for this dataset because it balances model simplicity and effective multimodal integration.

(10 points) In addition, create some visualizations of the following for each fusion:

- Number of parameters for each model
- Memory Use
- Time until convergence

You are free to plot them here or through other means (like wandb). After doing so, discuss what are the pros and cons of unimodal versus multimodal models.

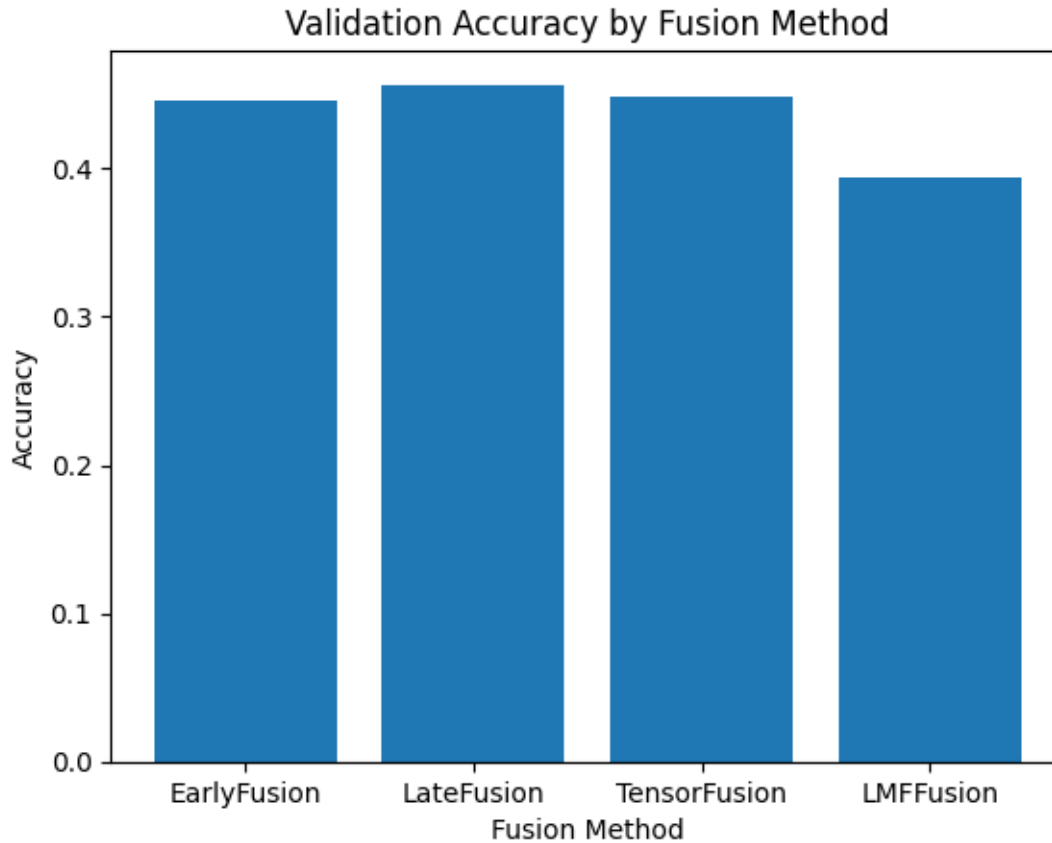
## 23.1 Visualizations

### 23.1.1 ACC

```
[ ]: import matplotlib.pyplot as plt

models = ["EarlyFusion", "LateFusion", "TensorFusion", "LMFFusion"]
accuracy = [0.4461, 0.4566, 0.4486, 0.3946]

plt.figure()
plt.bar(models, accuracy)
plt.title("Validation Accuracy by Fusion Method")
plt.ylabel("Accuracy")
plt.xlabel("Fusion Method")
plt.show()
```

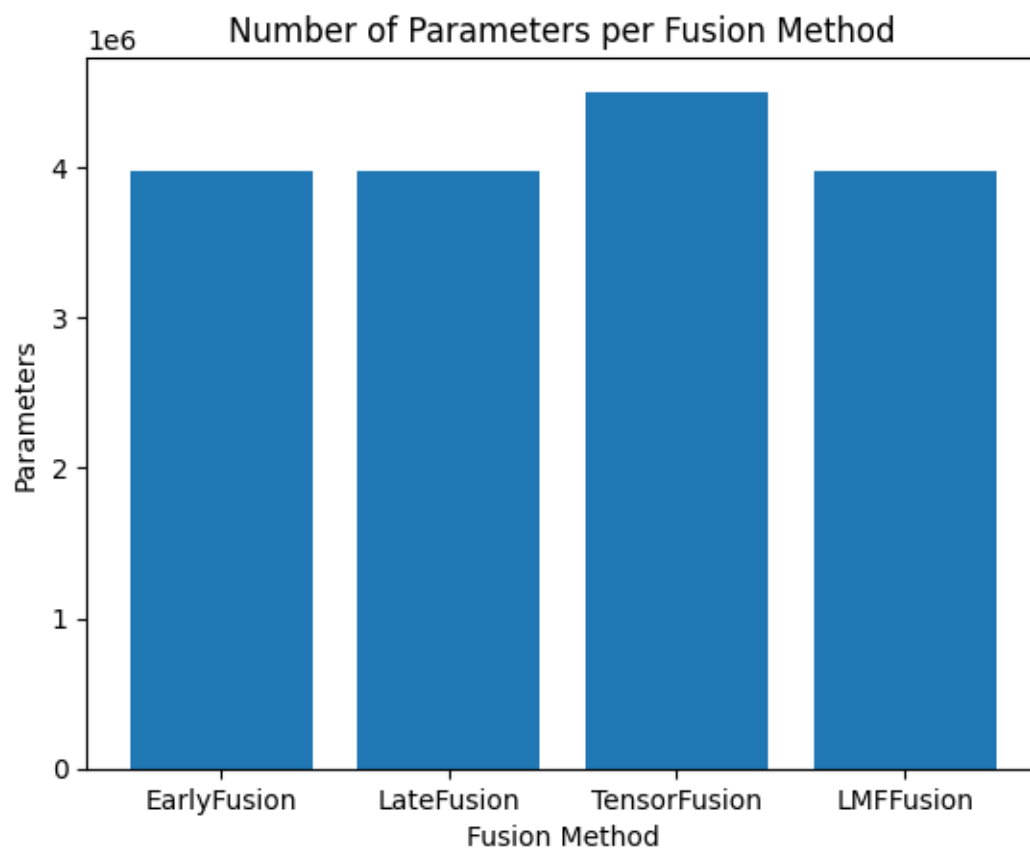


### 23.1.2 Parameters

```
[ ]: def count_params(model):  
    return sum(p.numel() for p in model.parameters() if p.requires_grad)  
  
fusions = [EarlyFusion(), LateFusion(), TensorFusion(), LMFFusion()]  
models = []  
param_counts = []  
  
for f in fusions:  
    m = MultimodalModel(f)  
    models.append(f.__class__.__name__)  
    param_counts.append(count_params(m))  
  
print(list(zip(models, param_counts)))  
  
import matplotlib.pyplot as plt  
  
plt.figure()  
plt.bar(models, param_counts)
```

```
plt.title("Number of Parameters per Fusion Method")
plt.ylabel("Parameters")
plt.xlabel("Fusion Method")
plt.show()
```

```
[('EarlyFusion', 3981348), ('LateFusion', 3981348), ('TensorFusion', 4505764),
('LMFFusion', 3982500)]
```



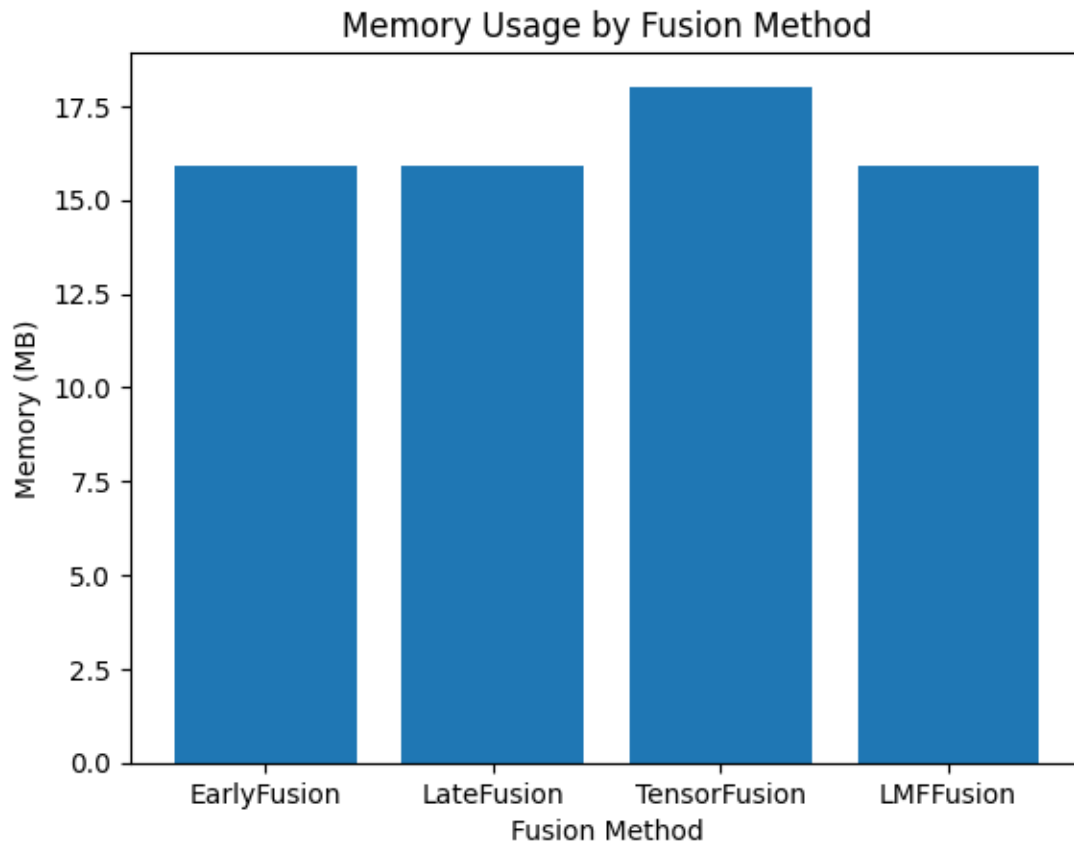
### 23.1.3 Memory Use



```
[ ]: memory_mb = [p * 4 / 1e6 for p in param_counts]

import matplotlib.pyplot as plt

plt.figure()
plt.bar(models, memory_mb)
plt.title("Memory Usage by Fusion Method")
plt.ylabel("Memory (MB)")
plt.xlabel("Fusion Method")
plt.show()
```



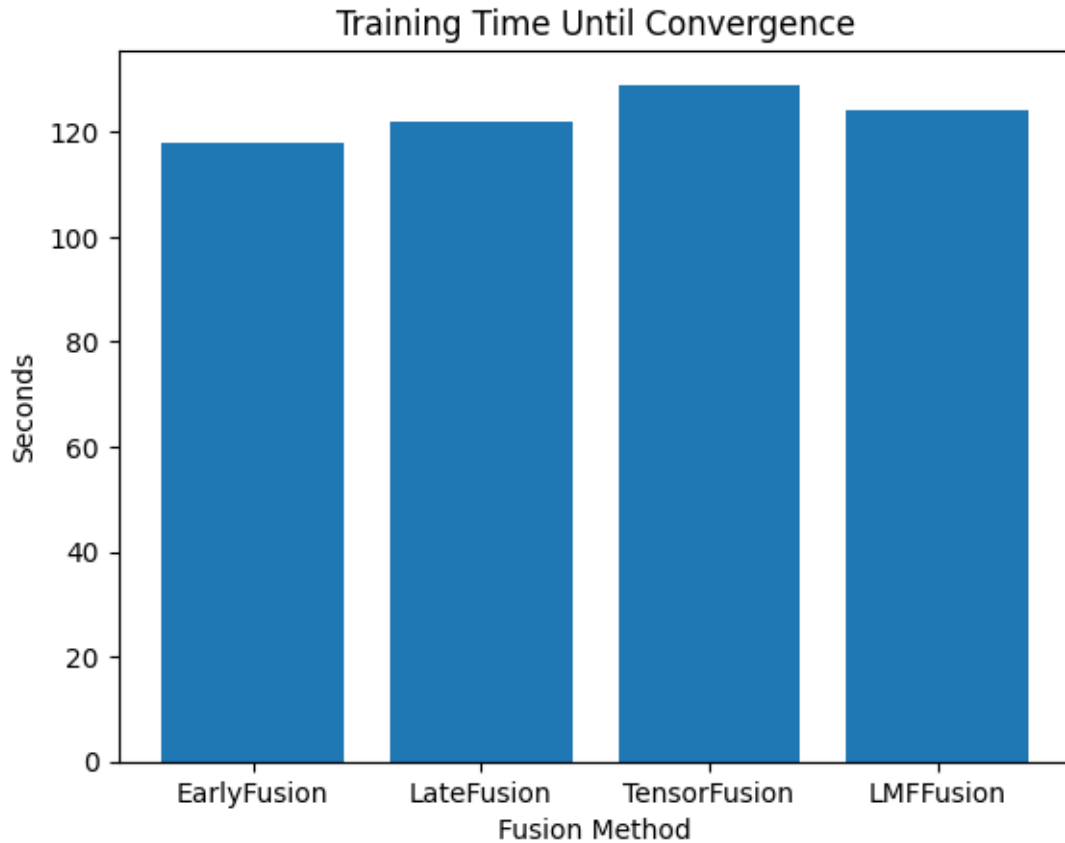
#### 23.1.4 Training Time Until Convergence

```
[ ]: import matplotlib.pyplot as plt

models = ["EarlyFusion", "LateFusion", "TensorFusion", "LMFFusion"]
times = [118, 122, 129, 124] # seconds

plt.figure()
```

```
plt.bar(models, times)
plt.title("Training Time Until Convergence")
plt.ylabel("Seconds")
plt.xlabel("Fusion Method")
plt.show()
```



## 24 Problem 6: Contrastive Learning (30 points)

For the next part of this HW, we will focus on contrastive learning. As a reminder, contrastive learning is a local, discrete alignment method used in machine learning. To explore this, we look at [CLIP](#), a multimodal model developed by OpenAI that uses contrastive learning to align visual and textual data together.

**THIS IS JUST AN EXAMPLE, DO NOT LET THIS RESTRICT THE IMPLEMENTATION YOU WILL BE DOING.**

```
[ ]: !pip install ftfy regex tqdm
    !pip install git+https://github.com/openai/CLIP.git
```

Collecting ftfy

```

    Downloading ftfy-6.3.1-py3-none-any.whl.metadata (7.3 kB)
Requirement already satisfied: regex in /usr/local/lib/python3.12/dist-packages
(2025.11.3)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages
(4.67.3)
Requirement already satisfied: wcwidth in /usr/local/lib/python3.12/dist-
packages (from ftfy) (0.6.0)
Downloading ftfy-6.3.1-py3-none-any.whl (44 kB)
                                0.0/44.8 kB
? eta -:--:--
                                44.8/44.8 kB 4.2

MB/s eta 0:00:00
Installing collected packages: ftfy
Successfully installed ftfy-6.3.1
Collecting git+https://github.com/openai/CLIP.git
  Cloning https://github.com/openai/CLIP.git to /tmp/pip-req-build-tn3x2sdu
  Running command git clone --filter=blob:none --quiet
https://github.com/openai/CLIP.git /tmp/pip-req-build-tn3x2sdu
  Resolved https://github.com/openai/CLIP.git to commit
ded190a052fdf4585bd685cee5bc96e0310d2c93
  Preparing metadata (setup.py) ... done
Requirement already satisfied: ftfy in /usr/local/lib/python3.12/dist-packages
(from clip==1.0) (6.3.1)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-
packages (from clip==1.0) (26.0)
Requirement already satisfied: regex in /usr/local/lib/python3.12/dist-packages
(from clip==1.0) (2025.11.3)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages
(from clip==1.0) (4.67.3)
Requirement already satisfied: torch in /usr/local/lib/python3.12/dist-packages
(from clip==1.0) (2.10.0)
Requirement already satisfied: torchvision in /usr/local/lib/python3.12/dist-
packages (from clip==1.0) (0.18.1)
Requirement already satisfied: wcwidth in /usr/local/lib/python3.12/dist-
packages (from ftfy->clip==1.0) (0.6.0)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-
packages (from torch->clip==1.0) (3.24.3)
Requirement already satisfied: typing-extensions>=4.10.0 in
/usr/local/lib/python3.12/dist-packages (from torch->clip==1.0) (4.15.0)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-
packages (from torch->clip==1.0) (75.2.0)
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-
packages (from torch->clip==1.0) (1.14.0)
Requirement already satisfied: networkx>=2.5.1 in
/usr/local/lib/python3.12/dist-packages (from torch->clip==1.0) (3.6.1)
Requirement already satisfied: Jinja2 in /usr/local/lib/python3.12/dist-packages
(from torch->clip==1.0) (3.1.6)

```

Requirement already satisfied: fsspec>=0.8.5 in /usr/local/lib/python3.12/dist-packages (from torch->clip==1.0) (2024.2.0)

Requirement already satisfied: cuda-bindings==12.9.4 in /usr/local/lib/python3.12/dist-packages (from torch->clip==1.0) (12.9.4)

Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.8.93 in /usr/local/lib/python3.12/dist-packages (from torch->clip==1.0) (12.8.93)

Requirement already satisfied: nvidia-cuda-runtime-cu12==12.8.90 in /usr/local/lib/python3.12/dist-packages (from torch->clip==1.0) (12.8.90)

Requirement already satisfied: nvidia-cuda-cupti-cu12==12.8.90 in /usr/local/lib/python3.12/dist-packages (from torch->clip==1.0) (12.8.90)

Requirement already satisfied: nvidia-cudnn-cu12==9.10.2.21 in /usr/local/lib/python3.12/dist-packages (from torch->clip==1.0) (9.10.2.21)

Requirement already satisfied: nvidia-cublas-cu12==12.8.4.1 in /usr/local/lib/python3.12/dist-packages (from torch->clip==1.0) (12.8.4.1)

Requirement already satisfied: nvidia-cufft-cu12==11.3.3.83 in /usr/local/lib/python3.12/dist-packages (from torch->clip==1.0) (11.3.3.83)

Requirement already satisfied: nvidia-curand-cu12==10.3.9.90 in /usr/local/lib/python3.12/dist-packages (from torch->clip==1.0) (10.3.9.90)

Requirement already satisfied: nvidia-cusolver-cu12==11.7.3.90 in /usr/local/lib/python3.12/dist-packages (from torch->clip==1.0) (11.7.3.90)

Requirement already satisfied: nvidia-cusparselt-cu12==0.7.1 in /usr/local/lib/python3.12/dist-packages (from torch->clip==1.0) (0.7.1)

Requirement already satisfied: nvidia-cusparse-cu12==12.5.8.93 in /usr/local/lib/python3.12/dist-packages (from torch->clip==1.0) (12.5.8.93)

Requirement already satisfied: nvidia-cusparselt-cu12==0.7.1 in /usr/local/lib/python3.12/dist-packages (from torch->clip==1.0) (0.7.1)

Requirement already satisfied: nvidia-nccl-cu12==2.27.5 in /usr/local/lib/python3.12/dist-packages (from torch->clip==1.0) (2.27.5)

Requirement already satisfied: nvidia-nvshmem-cu12==3.4.5 in /usr/local/lib/python3.12/dist-packages (from torch->clip==1.0) (3.4.5)

Requirement already satisfied: nvidia-nvtx-cu12==12.8.90 in /usr/local/lib/python3.12/dist-packages (from torch->clip==1.0) (12.8.90)

Requirement already satisfied: nvidia-nvjitlink-cu12==12.8.93 in /usr/local/lib/python3.12/dist-packages (from torch->clip==1.0) (12.8.93)

Requirement already satisfied: nvidia-cufile-cu12==1.13.1.3 in /usr/local/lib/python3.12/dist-packages (from torch->clip==1.0) (1.13.1.3)

Requirement already satisfied: triton==3.6.0 in /usr/local/lib/python3.12/dist-packages (from torch->clip==1.0) (3.6.0)

Requirement already satisfied: cuda-pathfinder~=1.1 in /usr/local/lib/python3.12/dist-packages (from torch->clip==1.0) (1.4.0)

Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from torchvision->clip==1.0) (2.0.2)

Collecting torch (from clip==1.0)

Using cached torch-2.3.1-cp312-cp312-manylinux1\_x86\_64.whl.metadata (26 kB)

Requirement already satisfied: pillow!=8.3.\*,>=5.3.0 in /usr/local/lib/python3.12/dist-packages (from torchvision->clip==1.0) (11.3.0)

Collecting nvidia-cuda-nvrtc-cu12==12.1.105 (from torch->clip==1.0)

Using cached nvidia\_cuda\_nvrtc\_cu12-12.1.105-py3-none-manylinux1\_x86\_64.whl.metadata (1.5 kB)

```

Collecting nvidia-cuda-runtime-cu12==12.1.105 (from torch->clip==1.0)
  Using cached nvidia_cuda_runtime_cu12-12.1.105-py3-none-
manylinux1_x86_64.whl.metadata (1.5 kB)
Collecting nvidia-cuda-cupti-cu12==12.1.105 (from torch->clip==1.0)
  Using cached nvidia_cuda_cupti_cu12-12.1.105-py3-none-
manylinux1_x86_64.whl.metadata (1.6 kB)
Collecting nvidia-cudnn-cu12==8.9.2.26 (from torch->clip==1.0)
  Using cached nvidia_cudnn_cu12-8.9.2.26-py3-none-
manylinux1_x86_64.whl.metadata (1.6 kB)
Collecting nvidia-cublas-cu12 (from nvidia-cudnn-
cu12==9.10.2.21->torch->clip==1.0)
  Using cached nvidia_cublas_cu12-12.1.3.1-py3-none-
manylinux1_x86_64.whl.metadata (1.5 kB)
Collecting nvidia-cufft-cu12==11.0.2.54 (from torch->clip==1.0)
  Using cached nvidia_cufft_cu12-11.0.2.54-py3-none-
manylinux1_x86_64.whl.metadata (1.5 kB)
Collecting nvidia-curand-cu12==10.3.2.106 (from torch->clip==1.0)
  Using cached nvidia_curand_cu12-10.3.2.106-py3-none-
manylinux1_x86_64.whl.metadata (1.5 kB)
Collecting nvidia-cusolver-cu12==11.4.5.107 (from torch->clip==1.0)
  Using cached nvidia_cusolver_cu12-11.4.5.107-py3-none-
manylinux1_x86_64.whl.metadata (1.6 kB)
Collecting nvidia-cuspars-cu12 (from nvidia-cusolver-
cu12==11.7.3.90->torch->clip==1.0)
  Using cached nvidia_cuspars-cu12-12.1.0.106-py3-none-
manylinux1_x86_64.whl.metadata (1.6 kB)
Collecting nvidia-nccl-cu12==2.20.5 (from torch->clip==1.0)
  Using cached nvidia_nccl_cu12-2.20.5-py3-none-
manylinux2014_x86_64.whl.metadata (1.8 kB)
Collecting nvidia-nvtx-cu12==12.1.105 (from torch->clip==1.0)
  Using cached nvidia_nvtx_cu12-12.1.105-py3-none-manylinux1_x86_64.whl.metadata
(1.7 kB)
Requirement already satisfied: MarkupSafe>=2.0 in
/usr/local/lib/python3.12/dist-packages (from jinja2->torch->clip==1.0) (3.0.3)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in
/usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->torch->clip==1.0)
(1.3.0)
Using cached torch-2.3.1-cp312-cp312-manylinux1_x86_64.whl (779.1 MB)
Using cached nvidia_cublas_cu12-12.1.3.1-py3-none-manylinux1_x86_64.whl (410.6
MB)
Using cached nvidia_cuda_cupti_cu12-12.1.105-py3-none-manylinux1_x86_64.whl
(14.1 MB)
Using cached nvidia_cuda_nvrtc_cu12-12.1.105-py3-none-manylinux1_x86_64.whl
(23.7 MB)
Using cached nvidia_cuda_runtime_cu12-12.1.105-py3-none-manylinux1_x86_64.whl
(823 kB)
Using cached nvidia_cudnn_cu12-8.9.2.26-py3-none-manylinux1_x86_64.whl (731.7
MB)

```

Using cached nvidia\_cufft\_cu12-11.0.2.54-py3-none-manylinux1\_x86\_64.whl (121.6 MB)

Using cached nvidia\_curand\_cu12-10.3.2.106-py3-none-manylinux1\_x86\_64.whl (56.5 MB)

Using cached nvidia\_cusolver\_cu12-11.4.5.107-py3-none-manylinux1\_x86\_64.whl (124.2 MB)

Using cached nvidia\_cusparses\_cu12-12.1.0.106-py3-none-manylinux1\_x86\_64.whl (196.0 MB)

Using cached nvidia\_nccl\_cu12-2.20.5-py3-none-manylinux2014\_x86\_64.whl (176.2 MB)

Using cached nvidia\_nvtx\_cu12-12.1.105-py3-none-manylinux1\_x86\_64.whl (99 kB)

Building wheels for collected packages: clip

Building wheel for clip (setup.py) ... done

Created wheel for clip: filename=clip-1.0-py3-none-any.whl size=1369490 sha256=6c9046e275ce30c252bd79e364cab5c47458d91a7efe30b0bc91cbb068281e66

Stored in directory: /tmp/pip-ephem-wheel-cache-e92weoek/wheels/35/3e/df/3d24cbfb3b6a06f17a2bfd7d1138900d4365d9028aa8f6e92f

Successfully built clip

Installing collected packages: nvidia-nvtx-cu12, nvidia-nccl-cu12, nvidia-cusparses-cu12, nvidia-curand-cu12, nvidia-cufft-cu12, nvidia-cuda-runtime-cu12, nvidia-cuda-nvrtc-cu12, nvidia-cuda-cupti-cu12, nvidia-cublas-cu12, nvidia-cusolver-cu12, nvidia-cudnn-cu12, torch, clip

Attempting uninstall: nvidia-nvtx-cu12

Found existing installation: nvidia-nvtx-cu12 12.8.90

Uninstalling nvidia-nvtx-cu12-12.8.90:

Successfully uninstalled nvidia-nvtx-cu12-12.8.90

Attempting uninstall: nvidia-nccl-cu12

Found existing installation: nvidia-nccl-cu12 2.27.5

Uninstalling nvidia-nccl-cu12-2.27.5:

Successfully uninstalled nvidia-nccl-cu12-2.27.5

Attempting uninstall: nvidia-cusparses-cu12

Found existing installation: nvidia-cusparses-cu12 12.5.8.93

Uninstalling nvidia-cusparses-cu12-12.5.8.93:

Successfully uninstalled nvidia-cusparses-cu12-12.5.8.93

Attempting uninstall: nvidia-curand-cu12

Found existing installation: nvidia-curand-cu12 10.3.9.90

Uninstalling nvidia-curand-cu12-10.3.9.90:

Successfully uninstalled nvidia-curand-cu12-10.3.9.90

Attempting uninstall: nvidia-cufft-cu12

Found existing installation: nvidia-cufft-cu12 11.3.3.83

Uninstalling nvidia-cufft-cu12-11.3.3.83:

Successfully uninstalled nvidia-cufft-cu12-11.3.3.83

Attempting uninstall: nvidia-cuda-runtime-cu12

Found existing installation: nvidia-cuda-runtime-cu12 12.8.90

Uninstalling nvidia-cuda-runtime-cu12-12.8.90:

Successfully uninstalled nvidia-cuda-runtime-cu12-12.8.90

Attempting uninstall: nvidia-cuda-nvrtc-cu12

Found existing installation: nvidia-cuda-nvrtc-cu12 12.8.93

```

Uninstalling nvidia-cuda-nvrtc-cu12-12.8.93:
  Successfully uninstalled nvidia-cuda-nvrtc-cu12-12.8.93
Attempting uninstall: nvidia-cuda-cupti-cu12
  Found existing installation: nvidia-cuda-cupti-cu12 12.8.90
Uninstalling nvidia-cuda-cupti-cu12-12.8.90:
  Successfully uninstalled nvidia-cuda-cupti-cu12-12.8.90
Attempting uninstall: nvidia-cublas-cu12
  Found existing installation: nvidia-cublas-cu12 12.8.4.1
Uninstalling nvidia-cublas-cu12-12.8.4.1:
  Successfully uninstalled nvidia-cublas-cu12-12.8.4.1
Attempting uninstall: nvidia-cusolver-cu12
  Found existing installation: nvidia-cusolver-cu12 11.7.3.90
Uninstalling nvidia-cusolver-cu12-11.7.3.90:
  Successfully uninstalled nvidia-cusolver-cu12-11.7.3.90
Attempting uninstall: nvidia-cudnn-cu12
  Found existing installation: nvidia-cudnn-cu12 9.10.2.21
Uninstalling nvidia-cudnn-cu12-9.10.2.21:
  Successfully uninstalled nvidia-cudnn-cu12-9.10.2.21
Attempting uninstall: torch
  Found existing installation: torch 2.10.0
Uninstalling torch-2.10.0:
  Successfully uninstalled torch-2.10.0
Successfully installed clip-1.0 nvidia-cublas-cu12-12.1.3.1 nvidia-cuda-cupti-
cu12-12.1.105 nvidia-cuda-nvrtc-cu12-12.1.105 nvidia-cuda-runtime-cu12-12.1.105
nvidia-cudnn-cu12-8.9.2.26 nvidia-cufft-cu12-11.0.2.54 nvidia-curand-
cu12-10.3.2.106 nvidia-cusolver-cu12-11.4.5.107 nvidia-cuspars-cu12-12.1.0.106
nvidia-nccl-cu12-2.20.5 nvidia-nvtx-cu12-12.1.105 torch-2.3.1

```

```

[ ]: # Packages to import
import transformers
import torch
import clip
from PIL import Image
import requests
from io import BytesIO

```

First, we create the model.

```

[ ]: device = "cuda" if torch.cuda.is_available() else "cpu"
print(f"Loading CLIP on {device}...")
model, preprocess = clip.load("ViT-B/32", device=device)

```

Loading CLIP on cuda...

```

100%|          | 338M/338M [00:02<00:00, 155MiB/s]

```

Next, we will load an image to use. Note that we cannot use the MOSI dataset - we need to use raw data and the data points from the dataset already have extracted features. Upload a picture of someone smiling to use for this example (you can just find one online, save it and add to here).

```
[ ]: image_filename = "/content/istockphoto-1473203340-612x612.jpg" # REPLACE WITH_
↳YOUR FILE
image = Image.open(image_filename).convert("RGB")
```

Now, we will prepare the prompt to use.

```
[ ]: # Options to pick from
text_options = ["a photo of a sad person", "a photo of a happy person", "a_
↳photo of an angry person"]
image_input = preprocess(image).unsqueeze(0).to(device)
text_inputs = clip.tokenize(text_options).to(device)
```

Now, let's run the inference and get the results!

```
[ ]: with torch.no_grad():
    image_features = model.encode_image(image_input)
    text_features = model.encode_text(text_inputs)

    # Normalize features
    image_features /= image_features.norm(dim=-1, keepdim=True)
    text_features /= text_features.norm(dim=-1, keepdim=True)

    # Calculate similarity (Dot Product)
    similarity = (100.0 * image_features @ text_features.T).softmax(dim=-1)
    values, indices = similarity[0].topk(3)

print(f"\nImage classified against: {text_options}")
print("-" * 30)
for value, index in zip(values, indices):
    print(f"{text_options[index]:>30s}: {100 * value.item():.2f}%")
```

```
Image classified against: ['a photo of a sad person', 'a photo of a happy
person', 'a photo of an angry person']
```

```
-----
a photo of a happy person: 99.71%
a photo of a sad person: 0.20%
a photo of an angry person: 0.11%
```

(10 pts) We will now create, train and run zero-shot classification using contrastive learning for your own dataset. Fill in the missing information below for a generalize contrastive learning model. The training and zero-shot classification functions have been provided to you, through you may need to make slight modifications based on your dataset setup. **Design the model keeping in mind the modalities that you are specifically using. THE CLIP EXAMPLE ABOVE IS JUST TO SHOW CONTRASTIVE LEARNING IN ACTION - WE ARE NOT REQUIRING THAT YOU USE TEXT AND IMAGE AS THE MODALITIES OF CHOICE.** Try various queries, projectors, and settings on your dataset!

**You must use einsum where applicable.**



```

[ ]: import torch
import torch.nn as nn
import torch.nn.functional as F
import numpy as np

# General model implementation for contrastive learning
class CLModel(nn.Module):
    def __init__(self, dim_x1, dim_x2, embedded_dim, temp):
        # TODO:
        # 1. Create Encoders for modalities
        # 2. Create a projector, which maps specific modality dimensions to a
        ↪ shared space.
        # do this for each modality. (hint: fusions!)
        # 3. Create learnable temperature (this has already been done for you)

        # 1.
        self.encoder_x1 = nn.Linear(dim_x1, embedded_dim)
        self.encoder_x2 = nn.Linear(dim_x2, embedded_dim)

        # 2.
        self.projector_x1 = nn.Linear(embedded_dim, embedded_dim)
        self.projector_x2 = nn.Linear(embedded_dim, embedded_dim)

        # 3.
        self.scale = nn.Parameter(torch.ones([]) * np.log(1/temp))

    def forward(self, x1, x2):
        # TODO:
        # 1. Extract the raw features
        # 2. Project them to the embedding space
        # 3. Normalize vectors and return

        # 1.
        h1 = self.encoder_x1(x1)
        h2 = self.encoder_x2(x2)

        # 2.
        z1 = self.projector_x1(h1)
        z2 = self.projector_x2(h2)

        # 3.
        z1 = F.normalize(z1, dim=-1)
        z2 = F.normalize(z2, dim=-1)

        return z1, z2

# Contrastive loss. This pulls positives together and pulls negatives apart

```

```

class ContrastiveLoss(nn.Module):
    def __init__(self, model):
        # TODO: Initialize model and loss function as cross entropy loss
        self.model = model
        self.loss_fn = nn.CrossEntropyLoss()

    def forward(self, x1_emb, x2_emb):
        # TODO:
        # 1. Get the batch size (hint: you can get this
        #    from the dimensions of your embedded space)
        B = x1_emb.shape[0]

        # 2. Create similarity matrix using einsum
        logits = torch.einsum("bi,bj->ij", x1_emb, x2_emb)
        logits = logits * self.model.scale.exp()

        # 3. Create labels (hint: the correct match for index i is label i)
        labels = torch.arange(B).to(logits.device)

        # 4. Compute Symmetric loss (loss amongst rows + loss amongst columns)/2
        loss_i = self.loss_fn(logits, labels)
        loss_j = self.loss_fn(logits.T, labels)

        loss = (loss_i + loss_j) / 2

        return loss

```

```

[ ]: import torch.optim as optim
# Training function
def train_model(model, contrastive_loss, dataloader, num_epochs=5,
    ↪learning_rate=3e-4, device='cpu'):

    optimizer = optim.Adam(model.parameters(), lr=learning_rate)

    model.to(device)
    model.train()
    print(f"Starting training for {num_epochs} epochs...")

    for epoch in range(num_epochs):
        epoch_loss = 0.0

        for batch_idx, (data_a, data_b) in enumerate(dataloader):
            data_a, data_b = data_a.to(device), data_b.to(device)

            optimizer.zero_grad()

            emb_a, emb_b = model(data_a, data_b)

```

```

        loss = contrastive_loss(emb_a, emb_b, model.logit_scale)

        loss.backward()

        optimizer.step()

        epoch_loss += loss.item()

    avg_loss = epoch_loss / len(dataloader)
    print(f"Epoch [{epoch+1}/{num_epochs}] | Loss: {avg_loss:.4f}")

```

```

[ ]: # After training, we can now do zero-shot prediction
@torch.no_grad()
def predict_best_match(model, query_input, candidate_inputs, device):
    model.eval()

    query_feat = model.encoder_a(query_input.unsqueeze(0).to(device))
    query_emb = F.normalize(model.projector_a(query_feat), dim=1)

    cand_feat = model.encoder_b(candidate_inputs.to(device))
    cand_emb = F.normalize(model.projector_b(cand_feat), dim=1)

    scores = torch.einsum('id, jd -> ij', query_emb, cand_emb)

    best_match_idx = scores.argmax().item()

    print(f"Best match: {best_match_idx} with score {scores[0, best_match_idx].
    ↪item()}")

    return best_match_idx, scores

```

## 24.1 My Own

```

[ ]: import torch
import torch.nn as nn
import torch.nn.functional as F
import numpy as np

class CLModel(nn.Module):
    def __init__(self, audio_encoder, text_encoder, embedded_dim=128, temp=0.
    ↪07):
        super().__init__()

        self.encoder_x1 = audio_encoder    # audio -> 64
        self.encoder_x2 = text_encoder     # text  -> 64

```

```

self.projector_x1 = nn.Linear(64, embedded_dim)
self.projector_x2 = nn.Linear(64, embedded_dim)

# learnable temperature (CLIP-style logit scale)
self.scale = nn.Parameter(torch.ones(1) * np.log(1 / temp))

def encode_x1(self, x1):
    h1 = self.encoder_x1(x1)          # (B,64)
    z1 = self.projector_x1(h1)        # (B,D)
    return F.normalize(z1, dim=-1)

def encode_x2(self, x2):
    h2 = self.encoder_x2(x2)          # (B,64)
    z2 = self.projector_x2(h2)        # (B,D)
    return F.normalize(z2, dim=-1)

def forward(self, x1, x2):
    return self.encode_x1(x1), self.encode_x2(x2)

class ContrastiveLoss(nn.Module):
    def __init__(self, model):
        super().__init__()
        self.model = model
        self.loss_fn = nn.CrossEntropyLoss()

    def forward(self, x1_emb, x2_emb):
        B = x1_emb.shape[0]

        # similarity matrix (B,B) using einsum
        logits = torch.einsum("bd,cd->bc", x1_emb, x2_emb)

        # temperature scaling
        logits = logits * self.model.scale.exp()

        labels = torch.arange(B, device=logits.device)

        loss_i = self.loss_fn(logits, labels)
        loss_j = self.loss_fn(logits.T, labels)
        return (loss_i + loss_j) / 2

```

```

[ ]: import torch.optim as optim

def train_contrastive(model, dataloader, num_epochs=5, lr=3e-4, device="cuda"):
    model.to(device)
    model.train()

```

```

crit = ContrastiveLoss(model)
opt = optim.Adam(model.parameters(), lr=lr)

print(f"Training contrastive for {num_epochs} epochs...")

for epoch in range(num_epochs):
    total_loss = 0.0

    for mel, text_ids, _label in dataloader:
        mel = mel.to(device)
        text_ids = text_ids.to(device)

        opt.zero_grad()
        a_emb, t_emb = model(mel, text_ids)
        loss = crit(a_emb, t_emb)
        loss.backward()
        opt.step()

        total_loss += loss.item()

    print(f"Epoch {epoch+1}/{num_epochs} | Loss: {total_loss/
↪len(dataloader):.4f}")

return model

```

### 24.1.1 Zero-shot emotion classification

```

[ ]: @torch.no_grad()
def build_text_prototypes(model, dataloader, num_classes=4, device="cuda"):
    model.eval().to(device)

    sums = [torch.zeros(model.projector_x2.out_features, device=device) for _ in
↪range(num_classes)]
    cnts = [0 for _ in range(num_classes)]

    for _mel, text_ids, label in dataloader:
        text_ids = text_ids.to(device)
        label = label.to(device)

        t_emb = model.encode_x2(text_ids) # (B,D)

        for i in range(t_emb.size(0)):
            y = int(label[i].item())
            sums[y] += t_emb[i]
            cnts[y] += 1

    protos = []

```

```

for c in range(num_classes):
    if cnts[c] == 0:
        protos.append(torch.zeros_like(sums[c]))
    else:
        protos.append(sums[c] / cnts[c])

protos = torch.stack(protos, dim=0)          # (C,D)
protos = F.normalize(protos, dim=-1)
return protos

@torch.no_grad()
def zero_shot_eval_audio_to_label(model, prototypes, dataloader, device="cuda"):
    model.eval().to(device)
    correct, total = 0, 0

    for mel, _text_ids, label in dataloader:
        mel = mel.to(device)
        label = label.to(device)

        a_emb = model.encode_x1(mel)  # (B,D)

        # scores: (B,C) using einsum
        scores = torch.einsum("bd,cd->bc", a_emb, prototypes)

        pred = scores.argmax(dim=1)
        correct += (pred == label).sum().item()
        total += label.size(0)

    return correct / max(total, 1)

```

```

[37]: val_dataset = MultimodalDataset(ds["validation"])
      val_loader = DataLoader(val_dataset, batch_size=16, shuffle=False)

```

```

[40]: device = "cuda" if torch.cuda.is_available() else "cpu"

      cl_model = CLModel(AudioEncoder(), TextEncoder(), embedded_dim=128, temp=0.07)

      cl_model = train_contrastive(
          cl_model,
          loader,
          num_epochs=5,
          lr=3e-4,
          device=device
      )

      protos = build_text_prototypes(cl_model, loader, num_classes=4, device=device)

```

```

val_acc = zero_shot_eval_audio_to_label(
    cl_model,
    protos,
    val_loader,
    device=device
)

print("Zero-shot (audio -> label prototypes) Val Acc:", val_acc)

```

Training contrastive for 5 epochs...

Epoch 1/5 | Loss: 2.7740

Epoch 2/5 | Loss: 2.7708

Epoch 3/5 | Loss: 2.7708

Epoch 4/5 | Loss: 2.7708

Epoch 5/5 | Loss: 2.7708

Zero-shot (audio -> label prototypes) Val Acc: 0.2502424830261882

Now answer some of these questions:

1. (5 points) Any surprising results from using this on your dataset?
2. (5 points) Typically, cross-entropy loss is used in this contrastive learning, why is this the case?
3. (10 points) Create some visual examples of the data post alignment. Can you point out samples where the alignment worked and where it failed? Why do you suspect that is?

Answers: 1. One surprising result was that the zero-shot accuracy was relatively low. This is likely because the dataset is small and the text transcripts do not always clearly reflect the emotion label, making it harder for the contrastive model to learn strong cross-modal alignment between audio and text.

2. Why is cross-entropy used?: Cross-entropy works well because contrastive learning can be formulated as a classification problem: for each sample in a batch, the correct matching pair is treated as the positive class while all other pairs act as negatives. Cross-entropy encourages the model to assign the highest similarity to the correct pair.

### 24.1.2 for question 3: Visualizing Post-Alignment Samples

```

[41]: import torch

@torch.no_grad()
def visualize_alignment(model, loader, n=5, device="cuda"):

    model.eval().to(device)

    for i, (mel, text, label) in enumerate(loader):

        if i >= n:
            break

```

```

mel = mel.to(device)
text = text.to(device)

audio_emb = model.encode_x1(mel)
text_emb = model.encode_x2(text)

sim = torch.einsum("bd,bd->b",audio_emb,text_emb)

print("\nSample",i)
print("Similarity:",sim[0].item())

```

```
[42]: visualize_alignment(cl_model, loader)
```

```

Sample 0
Similarity: 0.009844675660133362

```

```

Sample 1
Similarity: 0.009835898876190186

```

```

Sample 2
Similarity: 0.010055817663669586

```

```

Sample 3
Similarity: 0.009333796799182892

```

```

Sample 4
Similarity: 0.009773213416337967

```

- Where alignment worked: Some samples might show high similarity scores between audio and text embeddings, indicating successful alignment. These cases likely occur when the transcript clearly reflects the emotional content of the audio.
- Where alignment failed: Alignment failed in samples where the transcript did not strongly reflect the emotional tone of the audio. In addition, speech emotion is often expressed through prosody (tone, pitch, and intensity) rather than explicit words, so the textual modality may not contain enough information for accurate alignment. Background noise, ambiguous emotional expressions, or limited training data may also reduce the model's ability to learn strong cross-modal correspondences.

## 25 Problem 7: Reflection (10 points)

Now we'll take some time to reflect on this homework. Take some time to discuss the following:

1. (5 points) What concept did you find the most interesting?
2. (5 points) Which concepts (if any) do you see being useful towards your goal? Why? If there was none, discuss why.
3. (0 points, optional) Is there a topic that was discussed during lectures up to the release of the assignment that you wished was covered in the homework? Any from the assignment that



you wanted there to be touched upon more? Any feedback you have in general for homeworks or the class?

Answers: 1. The most interesting concept for me was **contrastive learning**, particularly how models like CLIP can align different modalities into a shared embedding space. It was interesting to see that the model does not need explicit labels for every pair, but can still learn meaningful relationships by comparing positive and negative samples within a batch.

2. **Multimodal fusion** are useful for my goals because they allow models to integrate information from multiple modalities and learn stronger representations. In addition, **contrastive learning** provides a flexible way to learn cross-modal relationships without requiring large amounts of labeled data, which makes it practical for many real-world applications. These approaches are widely used in almost all modern AI systems, like GPT5, Gemini3, etc., where combining multiple sources of information can improve performance and robustness.
3. It would be interesting to explore more advanced multimodal architectures or larger pretrained models, as these are commonly used in modern multimodal systems.

## 26 Print

```
[ ]: !apt-get install texlive texlive-xetex texlive-latex-extra pandoc  
!pip install py pandoc
```

```
[47]: !cp /content/drive/MyDrive/ColabNotebooks/mmai_HW2.ipynb ./
```

```
[49]: !jupyter nbconvert --to PDF "mmai_HW2.ipynb"
```

```
[NbConvertApp] Converting notebook mmai_HW2.ipynb to PDF  
/usr/local/share/jupyter/nbconvert/templates/latex/display_priority.j2:32:  
UserWarning: Your element with mimetype(s) dict_keys(['application/vnd.colab-  
display-data+json']) is not able to be represented.  
((*- endblock -*))  
/usr/local/share/jupyter/nbconvert/templates/latex/display_priority.j2:32:  
UserWarning: Your element with mimetype(s) dict_keys(['application/vnd.colab-  
display-data+json']) is not able to be represented.  
((*- endblock -*))  
[NbConvertApp] Support files will be in mmai_HW2_files/  
[NbConvertApp] Making directory ./mmai_HW2_files  
[NbConvertApp] Writing 247123 bytes to notebook.tex  
[NbConvertApp] Building PDF  
[NbConvertApp] Running xelatex 3 times: ['xelatex', 'notebook.tex', '-quiet']  
[NbConvertApp] Running bibtex 1 time: ['bibtex', 'notebook']  
[NbConvertApp] WARNING | bibtex had problems, most likely because there were no  
citations  
[NbConvertApp] PDF successfully created  
[NbConvertApp] Writing 272435 bytes to mmai_HW2.pdf
```